

NADAS (Natural & Artificial Disaster Alert System)

Project Team:

Ahmad Zakwan 24I-0505

Syed Faizan Haider 24I-0803

Muhammad Hadi 24I-0696

Course Instructor:

Ms. Afia Zafar

Course:

SDA (CS-3004)



Department of Computer Science

National University of Computer and Emerging Sciences

Islamabad, Pakistan

Table of Contents:

1.1	Purpose	3
1.2	Product Scope	3
1.3	Title	3
1.4	Objectives	4
1.5	Problem Statement	5
2	Overall Description	6
2.1	Product Perspective	6
2.2	Product Functions	6
2.3	List of Use Cases	8
2.4	Extended Use Cases	9
2.4.1	UC-01: Register for Hazard Alerts	9
2.4.2	UC-03: Report Local Hazard Condition.....	10
2.4.3	UC-05: Receive Hazard Alert Notification	11
2.4.4	UC-12: Configure System & Threshold Rules	12
2.5	Use Case Diagram	12
3	Other Non-Functional Requirements	13
3.1	Performance Requirements	13
3.2	Safety Requirements	13
3.3	Security Requirements	14
3.4	Software Quality Attributes	14
3.5	Business Rules.....	15
3.6	Operating Environment.....	15
3.7	User Interfaces	15
4	Domain Model	17
5	System Sequence Diagrams	20
5.1	SSD-01: Register for Hazard Alerts.....	20
5.2	SSD-02: Configure Alert Preferences	20
5.3	SSD-03: Report Local Hazard Condition.....	21
5.4	SSD-04: View Real-Time Hazard Dashboard.....	21
6	Sequence Diagrams	22
6.1	SD-01: Register for Hazard Alerts	22

6.2	SD-02: Configure Alert Preferences	22
6.3	SD-03: Report Local Hazard Condition.....	23
6.4	SD-04: View Real-Time Hazard Dashboard.....	23
6.5	SD-05: Receive Hazard Alert Notification.....	24
6.6	SD-06: Monitor Weather Data.....	24
6.7	SD-07: Monitor Disaster News Data	25
6.8	SD-12: Configure System & Threshold Rules	25
7	Class Diagram	26
7.1	Class Descriptions	26
7.1.1	Model Package (com.disaster.alert.model)	26
7.1.2	Service Package (com.disaster.alert.service).....	27
7.1.3	Database Package (com.disaster.alert.db).....	27
7.1.4	Utility Package (com.disaster.alert.util).....	27
8	Package Diagram	28
8.1	Package Structure.....	30
9	Deployment Diagram	31
9.1	Deployment Description.....	31
Appendix A: Design Patterns Applied		33
Appendix B: Technology Stack		35
A Project File Structure		36

1. Introduction

1.1 Purpose

This Software Requirements and Design Document (SRDD) specifies the complete re-quirements, architecture, design patterns, and implementation details for **NADAS** – the Natural and Artificial Disaster Alert System. This document covers the full software life-cycle from use case analysis through to database integration and serves as the definitive reference for the SDA (CS-3004) project evaluation at FAST NUCES Islamabad.

NADAS is a JavaFX 21 desktop application that enables local residents to receive real-time disaster alerts, report hazardous conditions with photo evidence, monitor live weather data, interact with a built-in disaster safety chatbot, and allows system administrators to review and approve hazard reports before official notifications are dispatched via email to all affected users.

1.2 Product Scope

NADAS addresses the critical need for timely, localised disaster information in Pakistan. The system integrates multiple real-time data sources including the USGS earthquake feed, WeatherAPI for current conditions and forecasts, and GDACS (Global Disaster Alert and Coordination System) for UN-backed disaster news. It provides a three-tier architecture comprising a JavaFX user interface, a Java business logic layer applying OOP principles and design patterns, and an SQLite embedded database for persistence.

The system is designed for three primary actors: Local Residents who register and receive alerts, Community Responders who monitor the real-time hazard dashboard, and System Administrators who manage reports and configure system thresholds.

1.3 Title

NADAS – Natural and Artificial Disaster Alert System: A real-time JavaFX desktop application providing integrated disaster monitoring, citizen hazard reporting

with photo evidence, an interactive earthquake heatmap, a full weather dashboard with animated effects, a disaster safety chatbot, and an admin portal for report approval and automated email notification dispatch – targeting Pakistani cities and powered by an embedded SQLite database.

1.4 Objectives

1. Provide real-time earthquake monitoring through the USGS GeoJSON API, displaying events on a Leaflet.js interactive map with heatmap overlay and circle markers.
2. Aggregate disaster news through a three-source fallback chain (GDACS → Re-liefWeb → USGS Significant) to ensure continuous data availability.
3. Enable citizens to register, log in, and configure personalised disaster alert preferences by city and hazard type (Weather, Earthquake, Fire, Flood).
4. Allow registered users to report local hazard conditions with description, location selection from 30 Pakistani cities, and optional photo evidence via FileChooser.
5. Implement a secure admin portal where administrators verify credentials, review colour-coded citizen reports, and approve or reject each report.
6. Automatically dispatch HTML email notifications via Gmail SMTP to registered users in the affected city upon admin approval; fall back to all users if no city match is found.
7. Display a dedicated Weather Dashboard with animated weather effects (rain, storm with lightning, snow, heat shimmer, wind, fog, clouds, sun), hourly and 3-day forecast, and Air Quality Index.
8. Provide a built-in Disaster Safety Chatbot with a local knowledge base covering earthquake, flood, fire, storm, landslide, first aid, heat wave, and Pakistan emergency numbers.
9. Maintain a comprehensive Alert History screen with full-text search, type filter, and severity filter backed by the SQLite database.

10. Apply GoF and GRASP design patterns (Singleton, Factory, Observer, Repository, MVC, Controller, Information Expert) throughout the system.

1.5 Problem Statement

Pakistan is one of the most disaster-prone countries in the world, frequently experiencing devastating earthquakes, floods, cyclones, and extreme weather events. The 2010 Indus River floods affected over 20 million people, and subsequent disasters have repeatedly displaced millions more. Despite the severity and frequency of these events, there remains no unified, accessible desktop application that combines real-time global disaster data with community-level reporting and direct email notification to affected residents.

Current information channels are fragmented: earthquake data requires visiting USGS or NDMA websites separately, weather warnings are distributed through SMS or television broadcasts, and community-level hazard reports have no formal submission or verification mechanism. Citizens in cities such as Islamabad, Rawalpindi, Lahore, Karachi, and Multan have no single platform to monitor all hazard types, report local conditions, or receive verified official alerts.

NADAS solves this problem by providing a unified desktop interface that aggregates real-time disaster data from multiple authoritative sources, enables citizen participation through hazard reporting with photo evidence, delivers verified information through automated HTML email notifications, and educates users through an integrated disaster safety chatbot – all within a three-layer MVC architecture backed by an embedded SQLite database that requires zero installation overhead.

2. Overall Description

2.1 Product Perspective

NADAS is a standalone desktop application built with JavaFX 21 and Java 21, following the Model-View-Controller (MVC) architectural pattern and organised into five clearly separated Java packages. It is a new, self-contained product that integrates with four external systems:

- **USGS Earthquake Hazards Program API** – real-time global earthquake Geo-JSON feed for the past 24 hours.
- **WeatherAPI.com** – current weather conditions, 3-day forecast, hourly forecast, UV index, dew point, visibility, and Air Quality Index for any city worldwide.
- **GDACS RSS Feed** – UN/European Commission disaster alert feed (XML for-mat); falls back automatically to UN ReliefWeb JSON, then USGS Significant Earthquakes if unavailable.
- **Gmail SMTP** – dispatching HTML email notifications to registered users in af-fected cities upon admin approval of citizen reports.

The system uses SQLite (nadas.db) as its embedded database for local persistence of users, admins, alerts, hazard reports, and threshold rules. The database file is created and seeded automatically on first application launch by the DatabaseManager Singleton class.

2.2 Product Functions

The major functional groups of NADAS are:

- **User Management:** Registration, login, profile management, password change, city and alert preference configuration.
- **Real-Time Data Monitoring:** Live earthquake data (USGS), weather data (WeatherAPI), disaster news (GDACS → ReliefWeb → USGS fallback chain).
- **Interactive Disaster Map:** Offline-capable world map with earthquake circle markers colour-coded by magnitude, heatmap intensity overlay, and approved citi-zen report pins in JavaFX WebView using Leaflet.js.
- **Weather Dashboard:** Dedicated full-screen weather screen with animated

weather canvas (rain, storm with lightning, snow, heat shimmer, wind, fog, clouds, sun), hourly forecast cards, 3-day forecast, and AQI breakdown.

- **Disaster Safety Chatbot:** Local knowledge-base chatbot covering 12 disaster categories with quick-chip buttons, accessible from the main dashboard.
- **Citizen Hazard Reporting:** Form with hazard type ComboBox, editable location ComboBox (30 Pakistani cities), description TextArea, optional photo FileChooser, and animated feedback.
- **Admin Portal:** Secure login with DB credential verification, colour-coded re-reports table (Pending/Approved/Rejected), photo preview panel, Approve/Reject buttons, and background email dispatch.
- **Alert History:** Searchable, filterable table of all alerts with type, severity, source, and timestamp columns.
- **Email Notification:** HTML emails via Gmail SMTP (App Password) to city-matched users upon admin approval; falls back to all users if no city match found.

2.3 List of Use Cases

UC#	Use Case Name	Primary Actor	Status
UC-01	Register for Hazard Alerts	Local Resident	Implemented
UC-02	Configure Alert Preferences	Local Resident	Implemented
UC-03	Report Local Hazard Condition	Local Resident	Implemented
UC-04	View Real-Time Hazard Dashboard	Local Resident	Implemented
UC-05	Receive Hazard Alert Notification	Local Resident	Implemented
UC-06	Monitor Weather Data	WeatherAPI / System	Implemented
UC-07	Monitor Disaster News Data	News API / System	Implemented
UC-08	Cluster & Validate Hazard Events	System	Implemented
UC-09	Predict Hazard Risk Level	System	Implemented
UC-10	Generate Impact Map	System	Implemented
UC-11	Issue Emergency Alert	System	Implemented
UC-12	Configure System & Threshold Rules	System Administrator	Implemented

Table 2.1: NADAS Use Case Summary

2.4 Extended Use Cases

2.4.1 UC-01: Register for Hazard Alerts

Field	Detail
Use Case Name	Register for
Hazard Alerts Scope	NADAS
Desktop Application	
Level	User-Goal Primary Actor Local Resident
Stakeholders	Local Resident (wants timely alerts); Admin (needs verified user records)
Pre-conditions	System online; internet available
Post-conditions	Resident account created in users table (SQLite)
Main Scenario	
1	Resident opens system; Login screen with particle animation is displayed.
2	Resident clicks “Register Now”.
3	System shows registration form.
4	Resident enters name, email, password, city, and alert types (checkboxes).
5	System validates via InputValidator.
6	Resident clicks “Create Account”.
7	AuthService.register() saves user to users table.
8	System navigates to Login screen after 1.2 seconds with success label.

2.1.1 UC-03: Report Local Hazard Condition

Field	Detail
Use Case Name	Report Local Hazard Condition
Primary Actor	Local Resident
Pre-conditions	Resident logged in (SessionManager.isLoggedIn() = true)
Post-conditions	Report saved in hazard_reports with status = 'Pending'
Main Scenario	
1	Resident clicks “My Reports” in navigation.
2	System displays ReportHazard.fxml form. City is pre-filled from user profile.
3	Resident selects hazard type from ComboBox (8 options).
4	Resident selects city from editable location ComboBox (30 cities).
5	Resident enters description (minimum 10 characters).
6	Resident optionally attaches a photo via FileChooser (JPG/P-NG/BMP).
7	Resident clicks “Submit Report”.
8	HazardReportService.saveReport() inserts row; returns generated ID.
9	Green glow success animation plays; form clears.
Extensions	
Empty fields: red shake animation shown. Description < 10 chars: specific validation error.	

Table 2.3: Extended Use Case – UC-03

2.4.2 UC-05: Receive Hazard Alert Notification

Field	Detail
Use Case Name	Receive Hazard Alert Notification
Primary Actor	Local Resident
Pre-conditions	User has email_alerts = 1; admin has approved a report
Post-conditions	HTML email delivered; alert saved to alerts table
Main Scenario	
1	Admin approves hazard report in Admin Dashboard.
2	AdminController.handleApprove() updates status to 'Approved'.
3	Alert object saved via AlertService.saveAlert().
4	Background thread calls DatabaseManager.getUserEmailsByCity(location).
5	If no city match, falls back to getAllUserEmails().
6	EmailService.sendAlertToUsers() sends HTML email via Gmail SMTP.
7	Subject: "NADAS ALERT – [HazardType] Warning in [Location]".
8	Admin UI status label updates with count of emails sent.
Extensions	
No city match: email sent to all users. Email failure: logged per-recipient to console.	

Table 2.4: Extended Use Case – UC-05

2.4.3 UC-12: Configure System & Threshold Rules

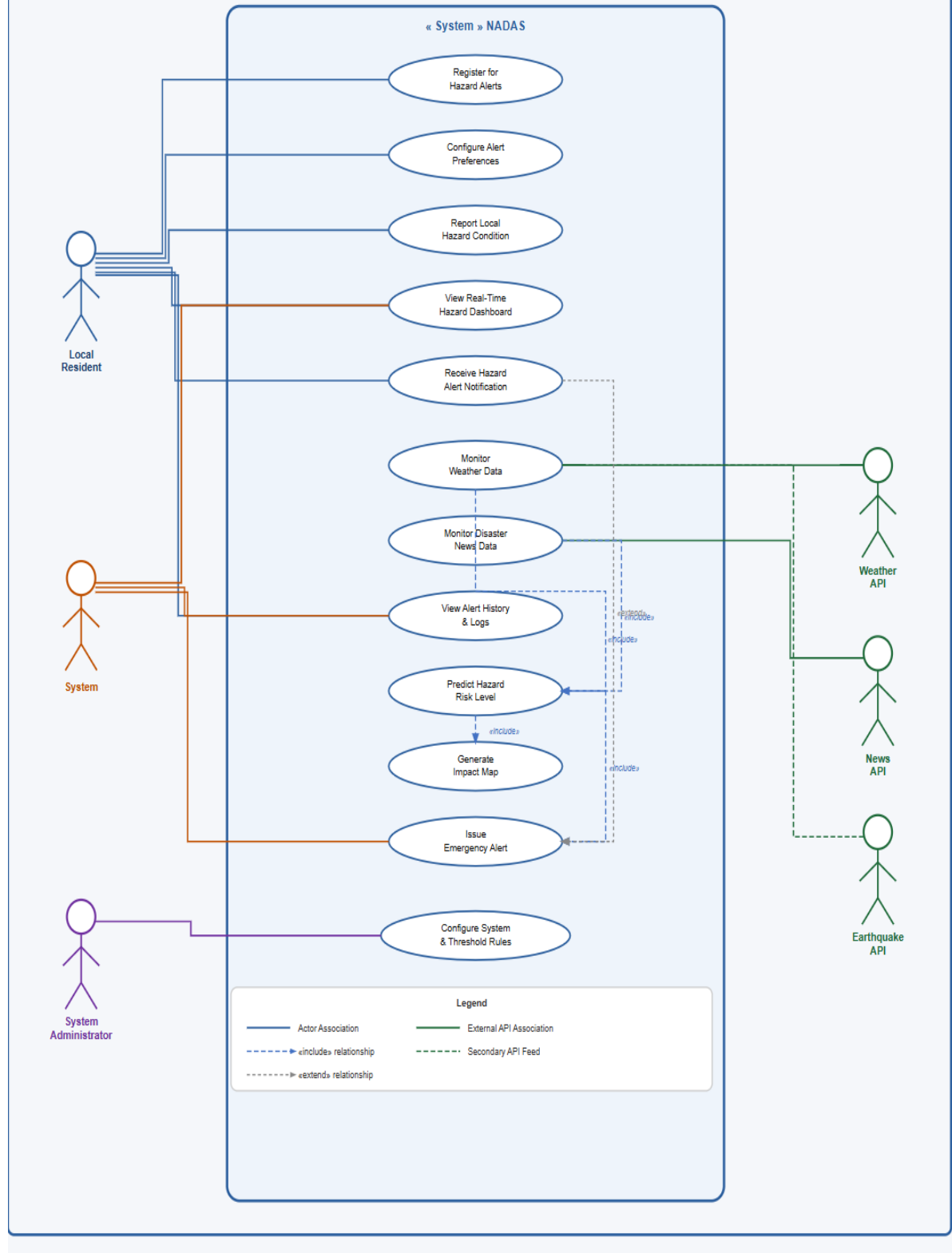
Field	Detail
Use Case Name	Configure System & Threshold Rules
Primary Actor	System Administrator
Pre-conditions	Admin authenticated via AdminLoginController
Post-conditions	Threshold rules updated in threshold_rules table
Main Scenario	
1	Admin enters credentials on AdminLogin.fxml.
2	LoginController calls DatabaseManager.verifyAdmin(email, password).
3	On success, navigates directly to Admin.fxml.
4	Admin views stats: total alerts, total reports, pending reports count.
5	Admin sees colour-coded reports table (yellow = Pending, green = Approved, r Rejected).
6	Admin clicks Approve ⇒ triggers email dispatch and map pin injection.
7	Admin clicks Reject ⇒ status updated; no alert or email issued.
8	Admin clicks Refresh ⇒ stats and table reload from DB.
Extensions	
Wrong credentials: “Invalid admin credentials. Access denied.” shown. Report already approved: inf Table 2.5: Extended Use Case – UC-12	

2.5 Use Case Diagram

NADAS — Natural & Artificial Disaster Alert System

Use Case Diagram | SDA (CS-3004)

Ahmad Zakwan — 241-0505
Syed Faizan Haider — 241-0803
Muhammad Hadi — 241-0696



3. Other Non-Functional Requirements

3.1 Performance Requirements

- API calls use `setConnectTimeout(7000)` and `setReadTimeout(7000)` to prevent UI freezing on slow connections.
- Weather, earthquake, and news data load in three independent background threads so no single slow API blocks the others.
- Email notifications are dispatched in a background thread via `new Thread(() -> {...}).start()` to keep the Admin UI responsive during bulk dispatch.
- `PreparedStatement` is used for all SQL operations, preventing SQL injection and improving query execution performance.
- The Leaflet.js map renders with `preferCanvas: true` for improved JavaFX WebView performance when displaying hundreds of earthquake markers.
- The Login screen particle network animation renders at 60 fps using JavaFX `AnimationTimer` with 80 particles.
- The system loads and displays dashboard data within 10 seconds on a standard internet connection.

3.2 Safety Requirements

- Admin access is protected by credential verification against the admins database table.
- The hazard reporting workflow includes mandatory admin approval before any alert is officially issued – no automatic alert dispatch without human review.
- All citizen reports are validated for minimum content requirements (non-empty location, description ≥ 10 characters) before database storage.

- False or spam reports can be rejected by admin, preventing unnecessary public alarm.
- Email credentials (App Password) are used instead of the primary account pass-word, limiting exposure if compromised.

3.3 Security Requirements

- Admin portal requires email and password authentication verified against the SQLite admins table via `DatabaseManager.verifyAdmin()`.
- Admin account is seeded by the system and cannot be registered through the public registration form (protected by `INSERT OR IGNORE`).
- User sessions are managed exclusively by the `SessionManager` Singleton, which holds the authenticated User object in memory.
- Input validation is performed via the `InputValidator` utility class before every database operation to prevent malformed data entry.
- Email notifications are sent only to users with `email_alerts = 1` in their profile, respecting user preferences.

3.4 Software Quality Attributes

- **Maintainability:** Strict MVC separation – controllers never write SQL; services never manipulate UI. Five distinct packages enforce boundaries.
- **Reliability:** Three-source fallback chain (GDACS → ReliefWeb → USGS Signifi-cant) ensures disaster news is always available.
- **Portability:** JavaFX 21 with SQLite (sqlite-jdbc 3.45.1) runs on Windows, macOS, and Linux without modification.
- **Reusability:** `SceneManager`, `SessionManager`, and `InputValidator` are utility singletons reused across all controllers.
- **Usability:** Particle network background, glassmorphism login card, green glow suc-cess animations, red shake error animations, and animated weather canvas provide professional UX feedback.
- **Extensibility:** `AlertFactory` uses the Factory pattern; new alert types can be added by extending the `AlertType` enum without modifying

existing code.

3.5 Business Rules

1. Only registered and logged-in users can submit hazard reports.
2. Hazard reports remain in Pending status until explicitly approved or rejected by an administrator.
3. Only Approved reports become official alerts visible in the Dashboard feed and map.
4. Email alerts are sent exclusively to users with `email_alerts = 1`.
5. The admin account is pre-seeded and cannot be created through the user registration flow.
6. Alert history stores all API-fetched alerts and admin-approved citizen reports for reference and audit.
7. The system must be connected to the internet to fetch live earthquake, weather, and news data; the offline map base layer remains functional without connectivity.

3.6 Operating Environment

Component	Specification
Platform	Windows 10/11, macOS 12+, Ubuntu 20.04+ Java Runtime JDK 21 (OpenJDK or Oracle JDK)
Build System	Apache Maven 3.8+ with <code>javafx-maven-plugin</code> 0.0.8
JavaFX	21.0.1 (<code>javafx-controls</code> , <code>javafx-fxml</code> , <code>javafx-web</code>)
Database	SQLite via <code>org.xerial:sqlite-jdbc:3.45.1.0</code>
(embedded) Email Library	<code>org.eclipse.angus:jakarta.mail:2.0.3</code>
(Jakarta Mail API) JSON Parsing	<code>org.json:json:20231013</code>
Map Library	Leaflet.js 1.9.4 (CDN) +

remains functional Table 3.1: Operating

Environment Specifications

3.7 User Interfaces

NADAS provides nine FXML-based screens, each with a corresponding JavaFX controller and CSS styling:

1. **Login.fxml** – Dark glassmorphism card; animated particle network background (ParticleNetworkCanvas); email/password fields; red shake error animation.
2. **Register.fxml** – Registration form: name, email, password, city ComboBox, alert type checkboxes; animated feedback.
3. **Dashboard.fxml** – Main screen: live Leaflet.js disaster map (WebView), weath-er/earthquake/news info cards, approved reports table, earthquake alert feed, map layer toggle buttons.
4. **ReportHazard.fxml** – Centred card; hazard type ComboBox (8 options); editable location ComboBox (30 cities); description TextArea; photo FileChooser; green glow success animation; red shake error animation.
5. **AlertHistory.fxml** – Full-text search field; type filter ComboBox; severity filter ComboBox; results count label; sortable table backed by SQLite alerts table.
6. **Settings.fxml** – Profile update (name, city); alert type checkboxes; severity radio buttons (Low/Medium/High); notification toggles; password change section.
7. **AdminLogin.fxml** – Restricted-access screen with admin credential validation and warning message.
8. **Admin.fxml** – Statistics cards (total alerts, total reports, pending count); colour-coded reports table; Approve/Reject buttons; photo preview panel.

Weather.fxml – Animated weather canvas; city search; hero section (icon, tem-perature, condition, tip); stats grid (wind, humidity, UV, dew, visibility, pressure); hourly forecast row; 3-day forecast cards; Air Quality Index panel.

2. Domain Model

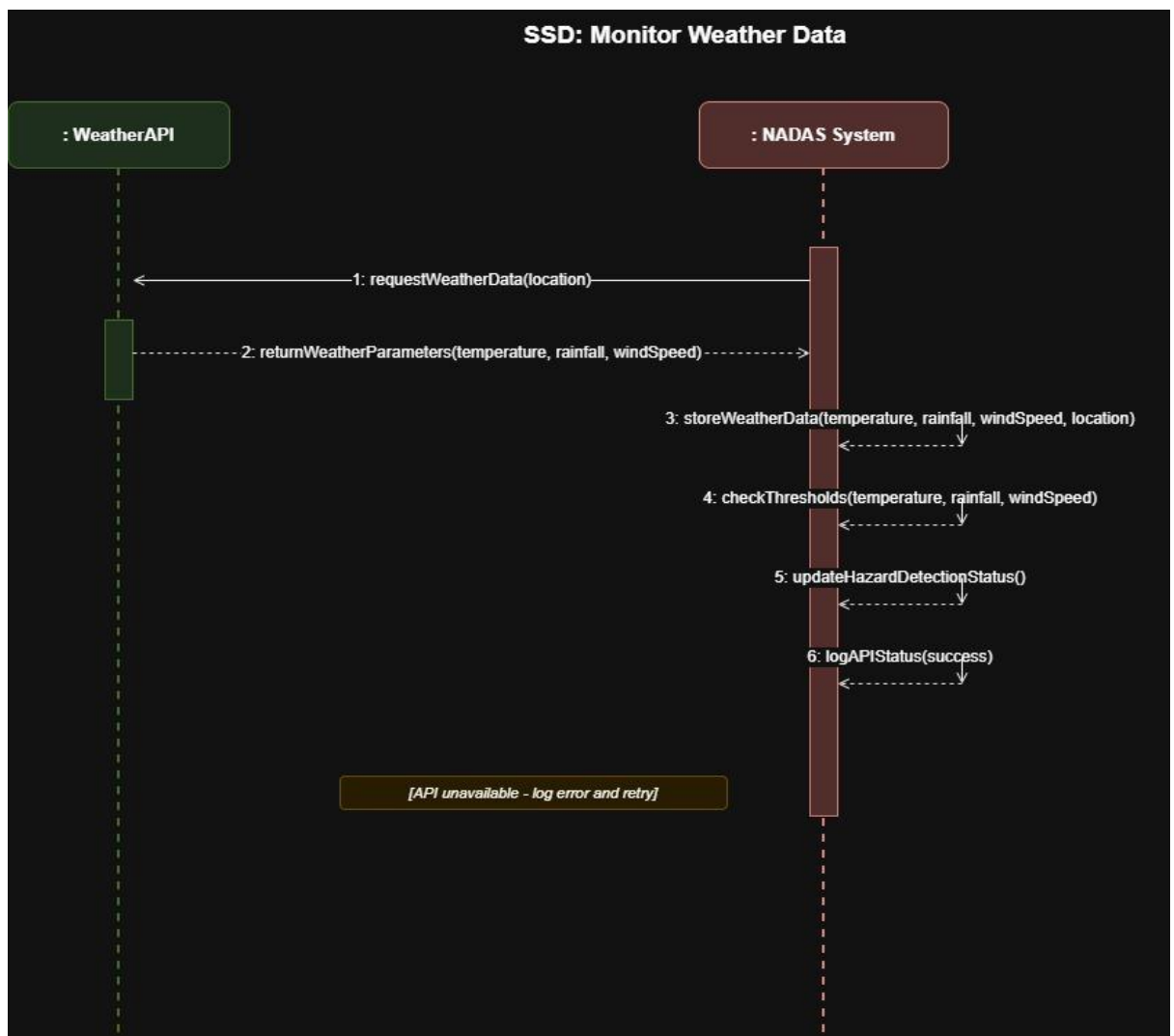
Version 3 — UML class diagram with proper multiplicity (1, *, 0..1, 1..*)



Figure 4.1: NADAS Domain Model Diagram

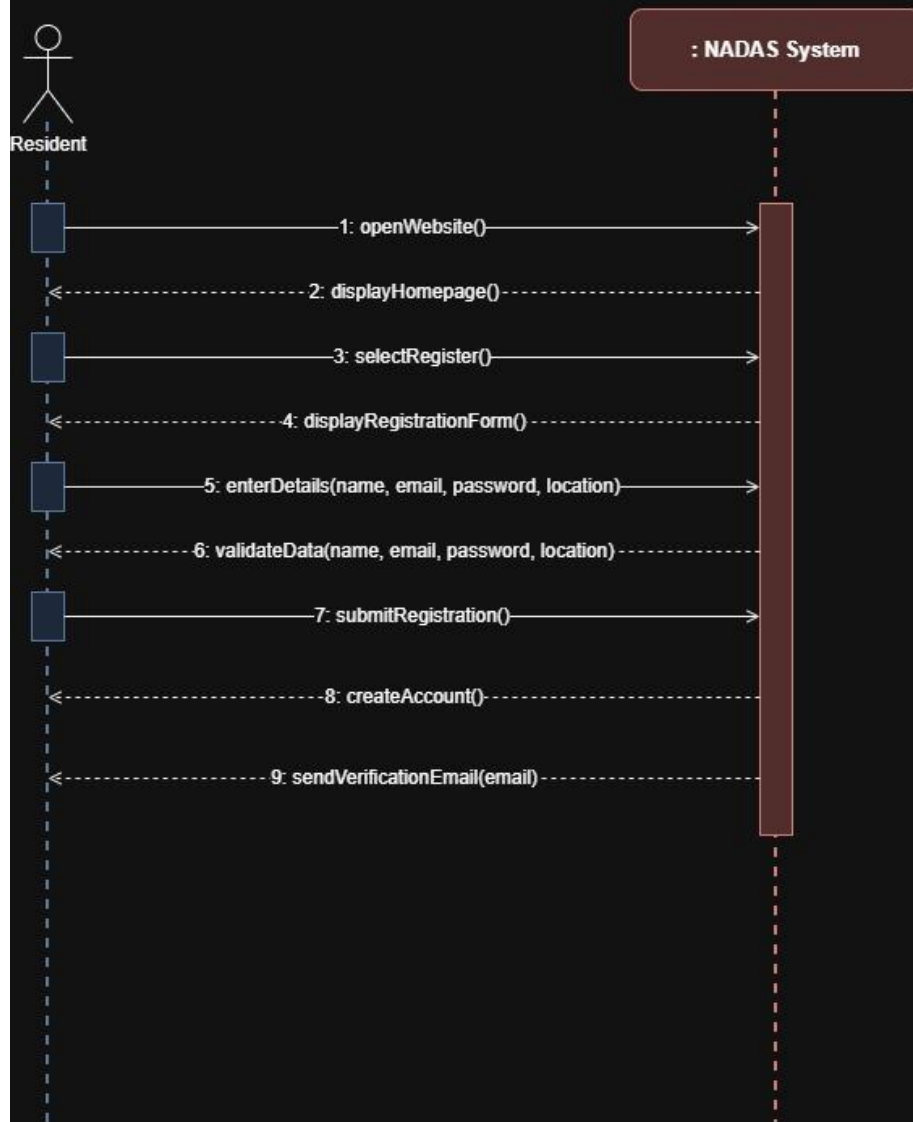
5. System Sequence Diagrams

1. Monitor Weather Data

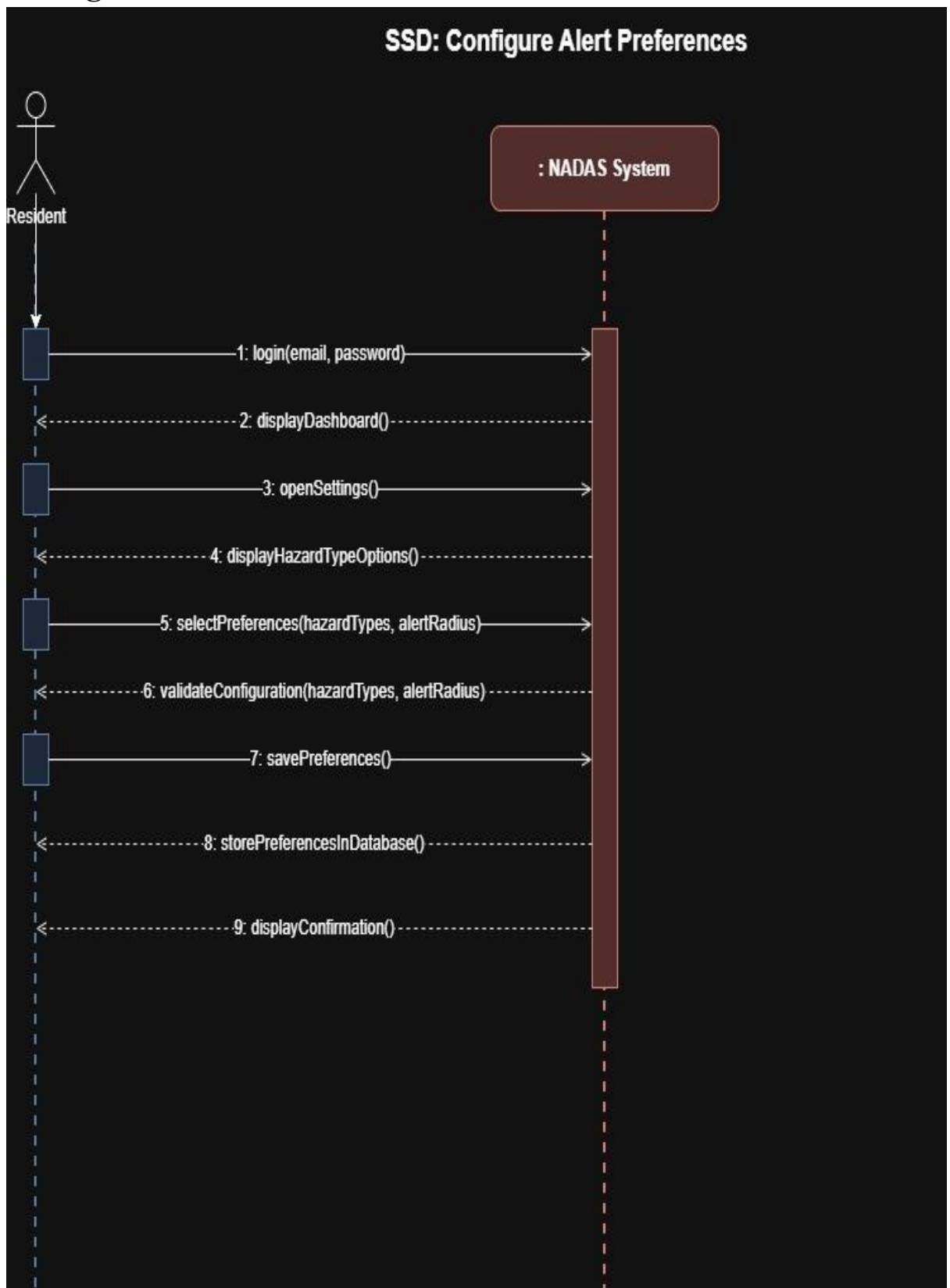


2. Register for Hazard Alerts

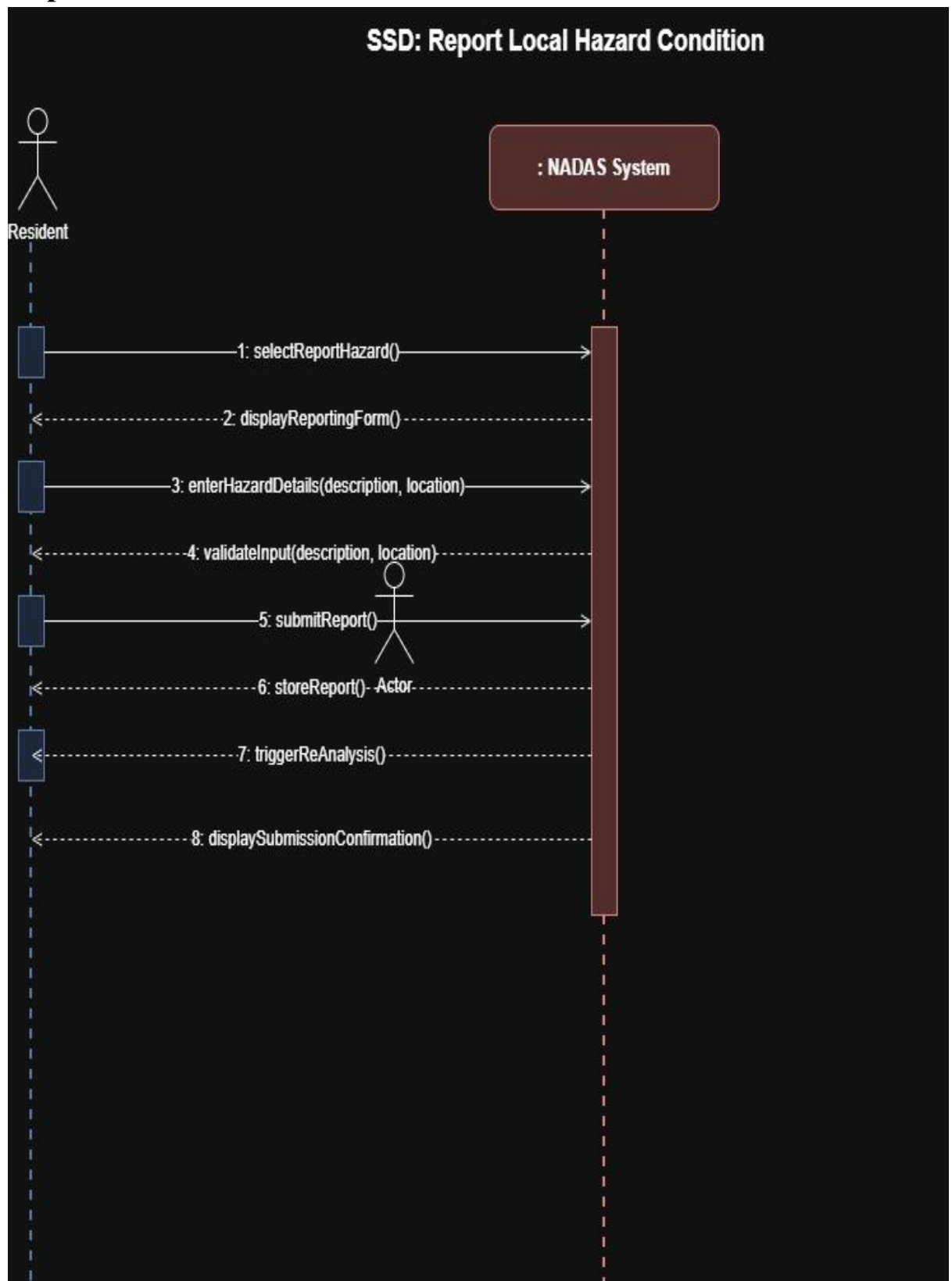
SSD: Register for Hazard Alerts



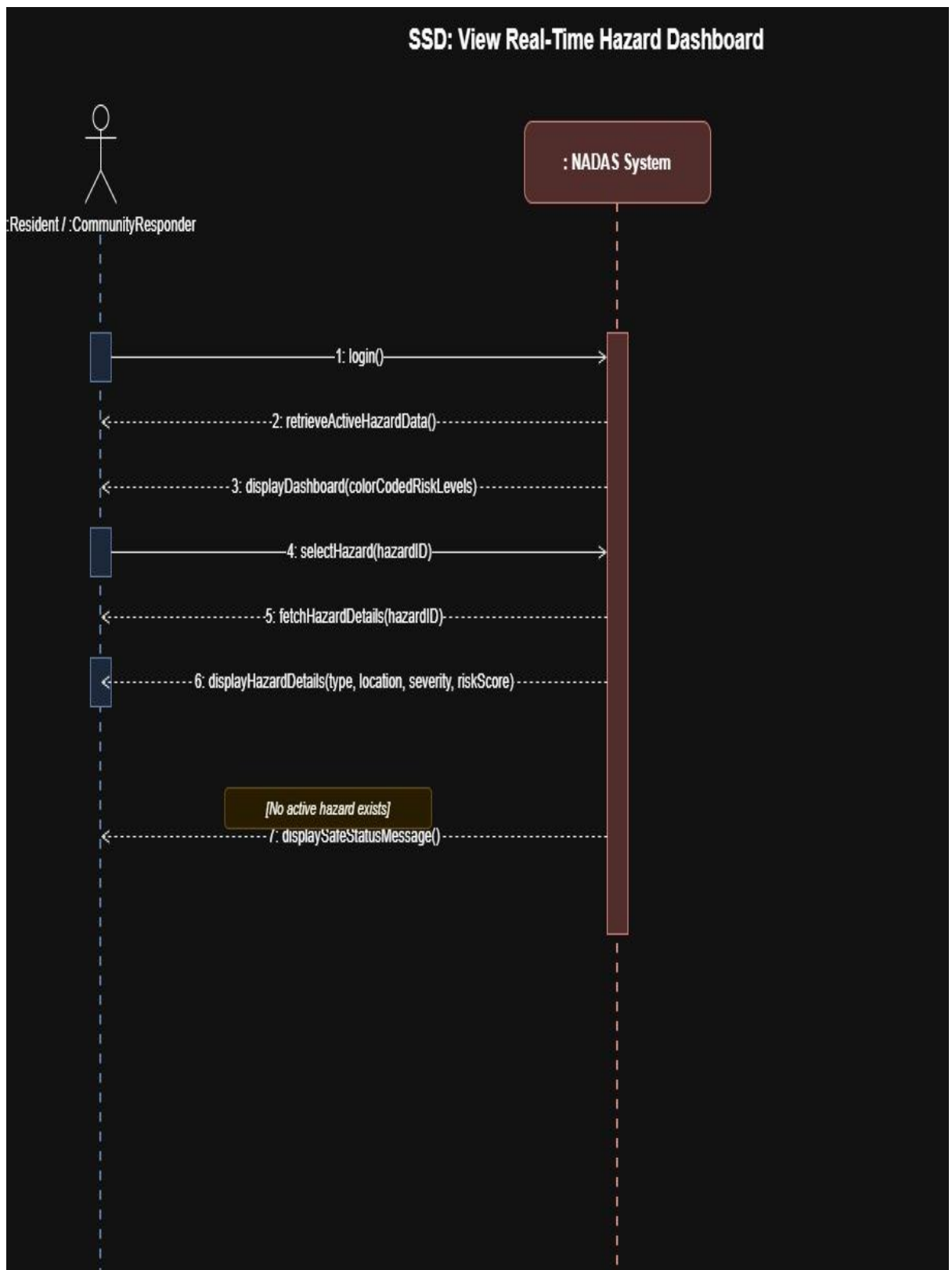
3. Configure Alert Preferences



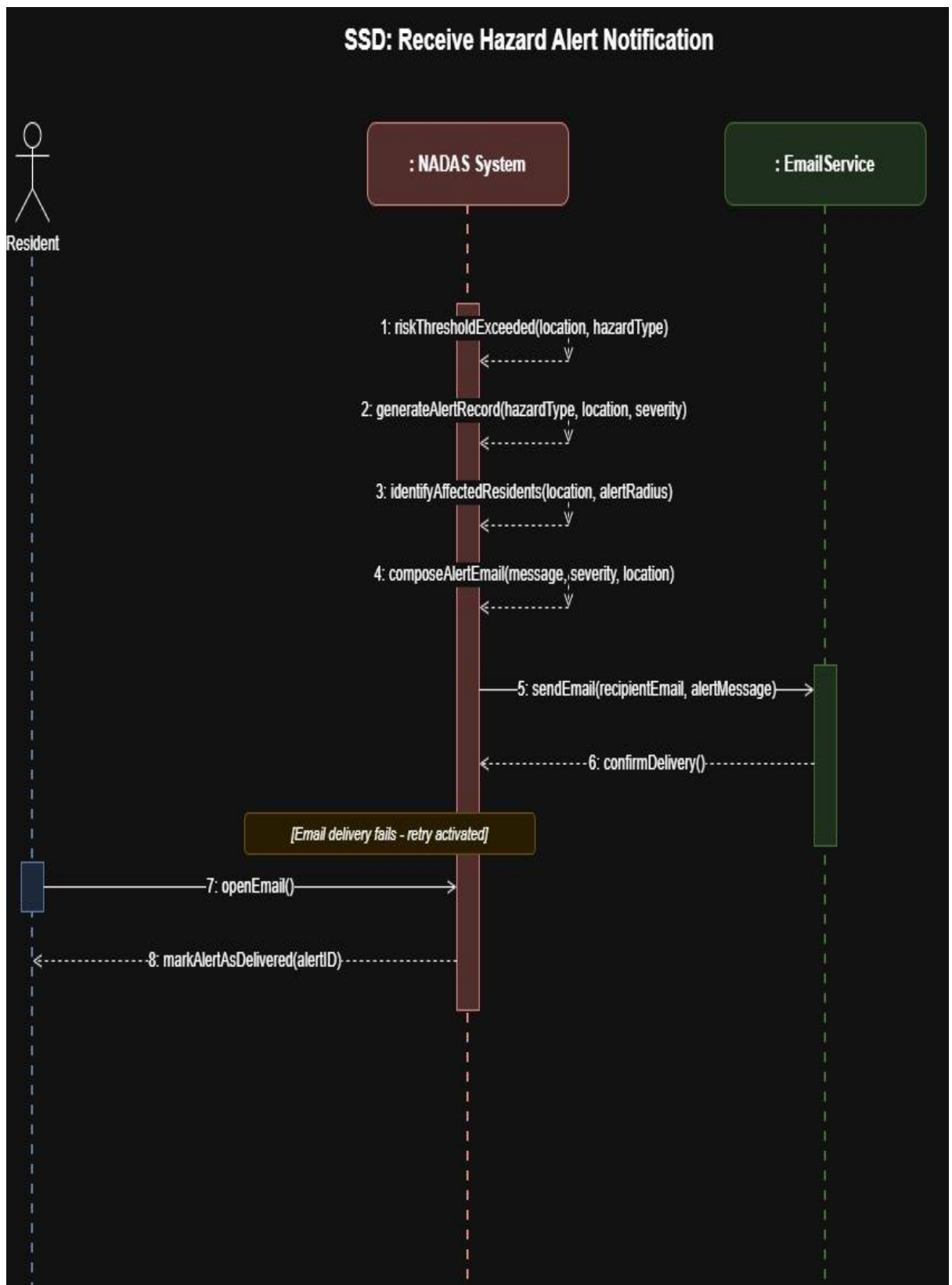
4. Report Local Hazard Condition



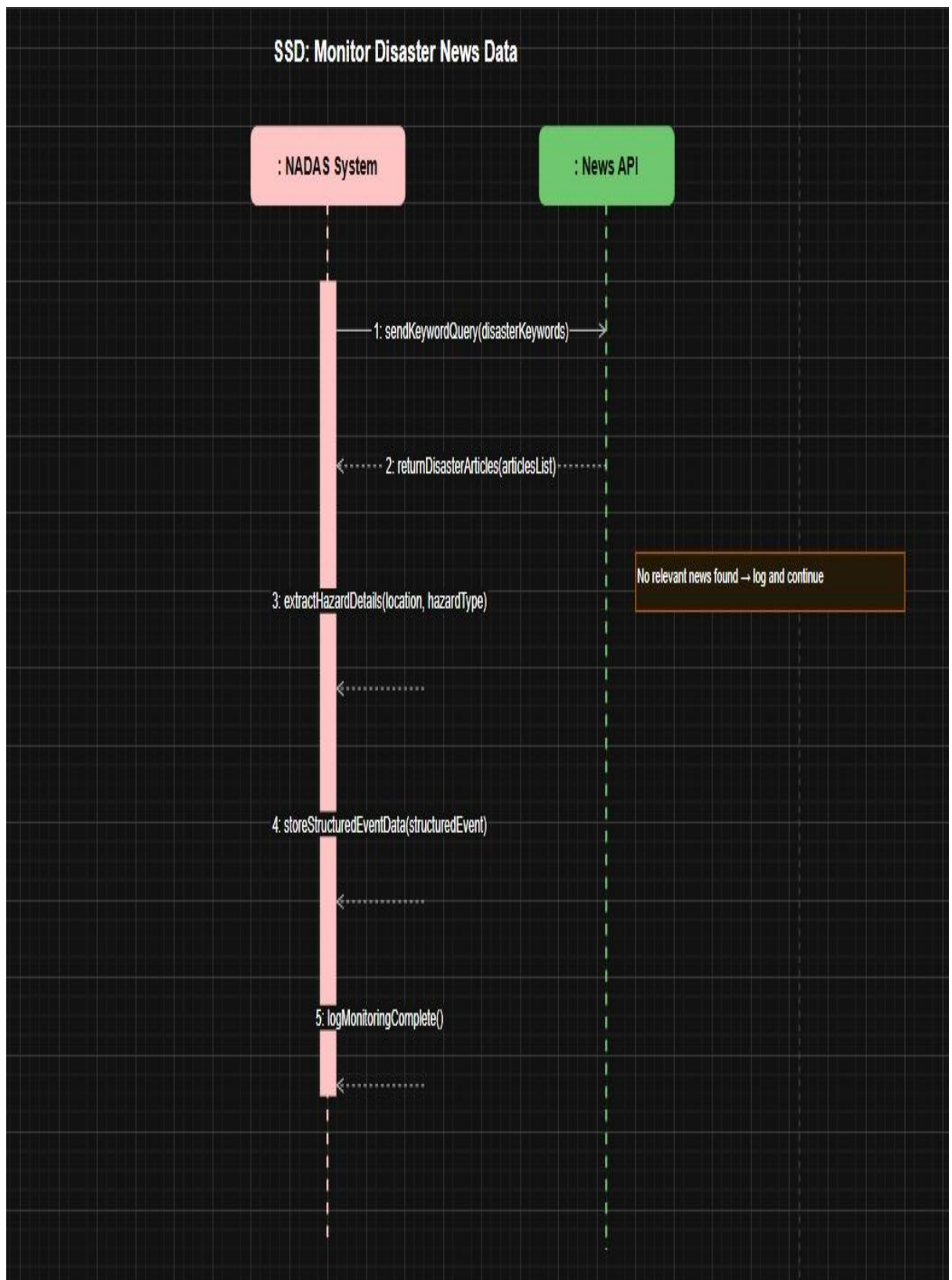
5. View Real-Time Hazard Dashboard



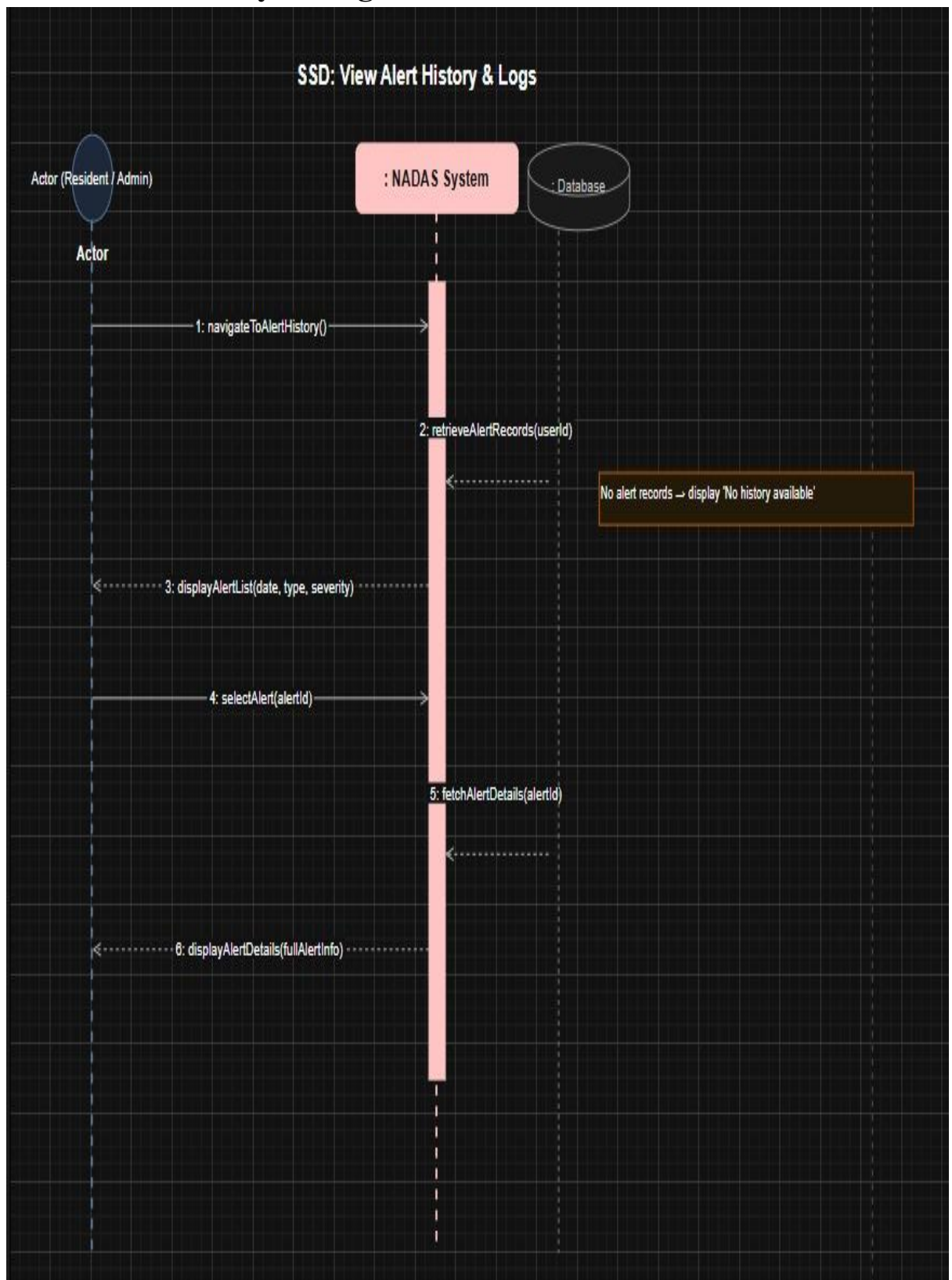
6. Receive Hazard Alert Notification



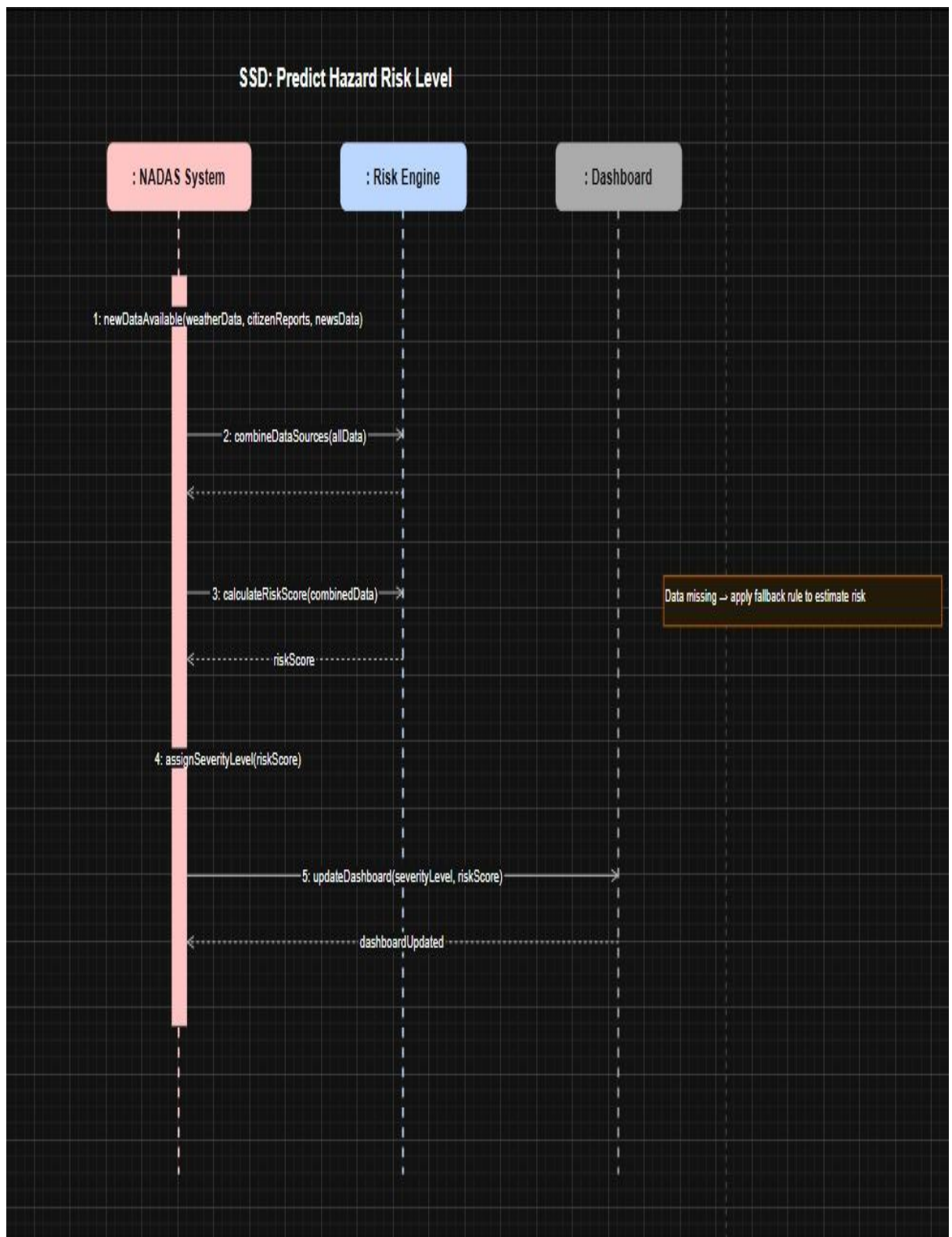
7. Monitor Disaster News Data



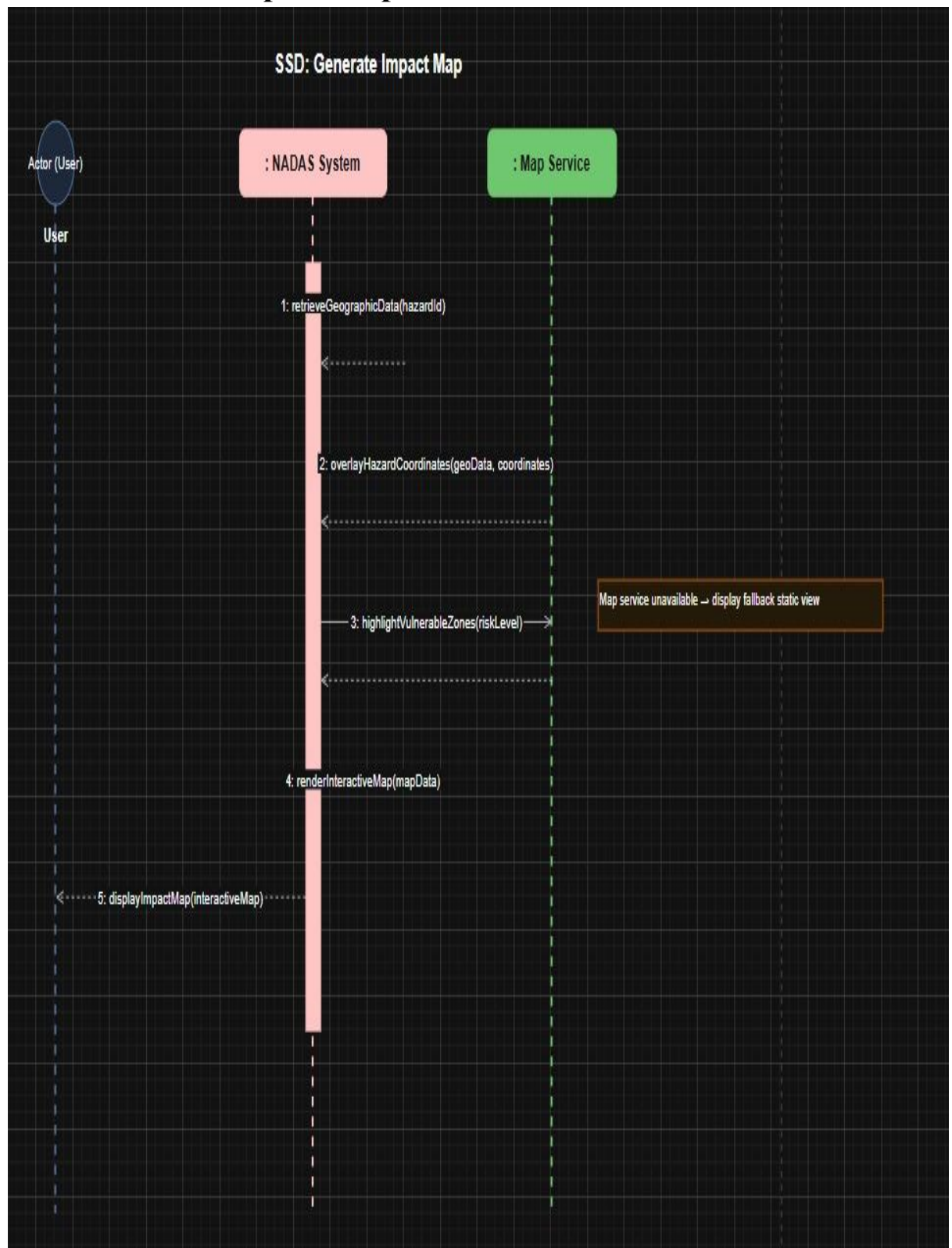
8. View Alert History & Logs



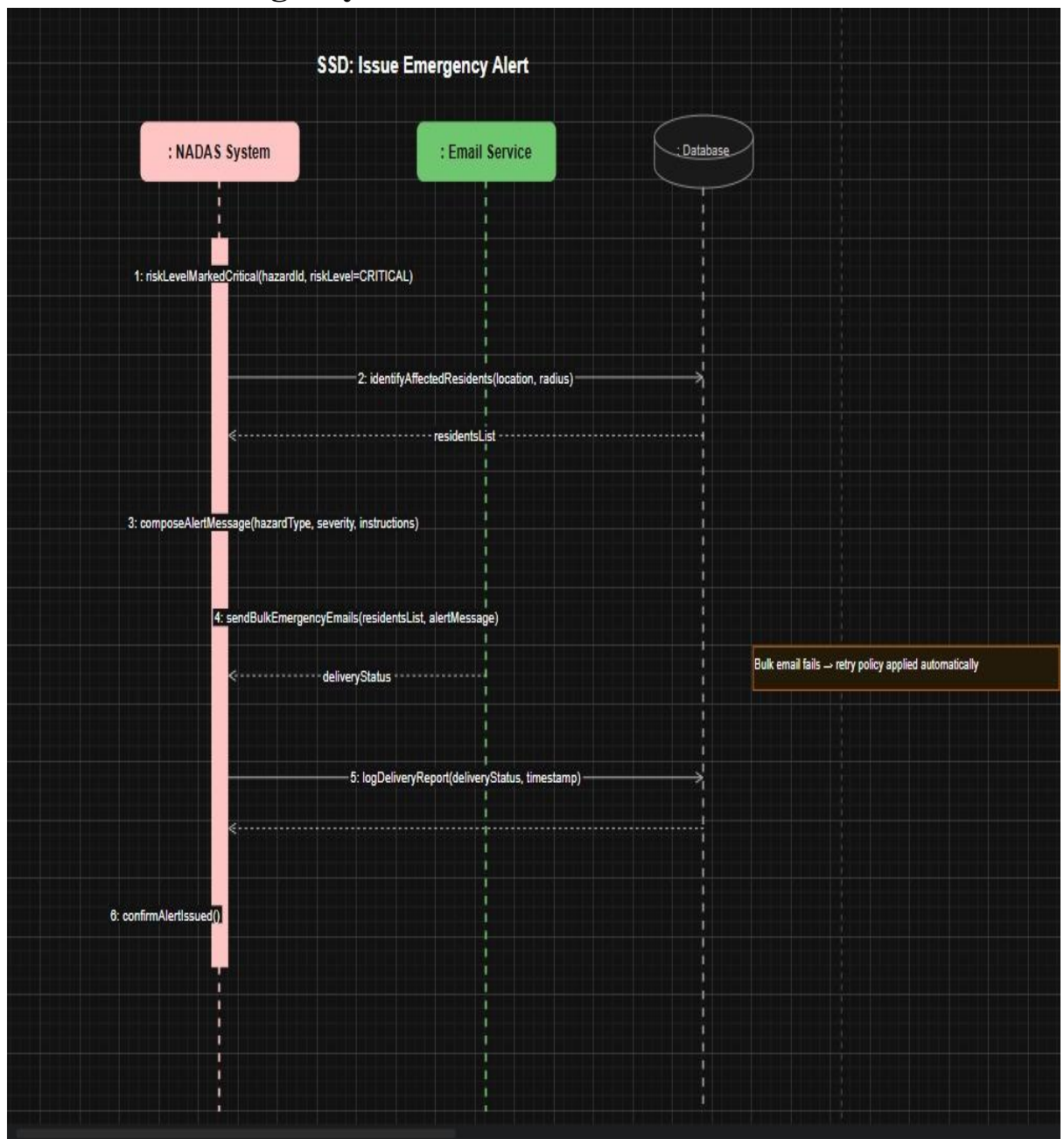
9. Predict Hazard Risk Level



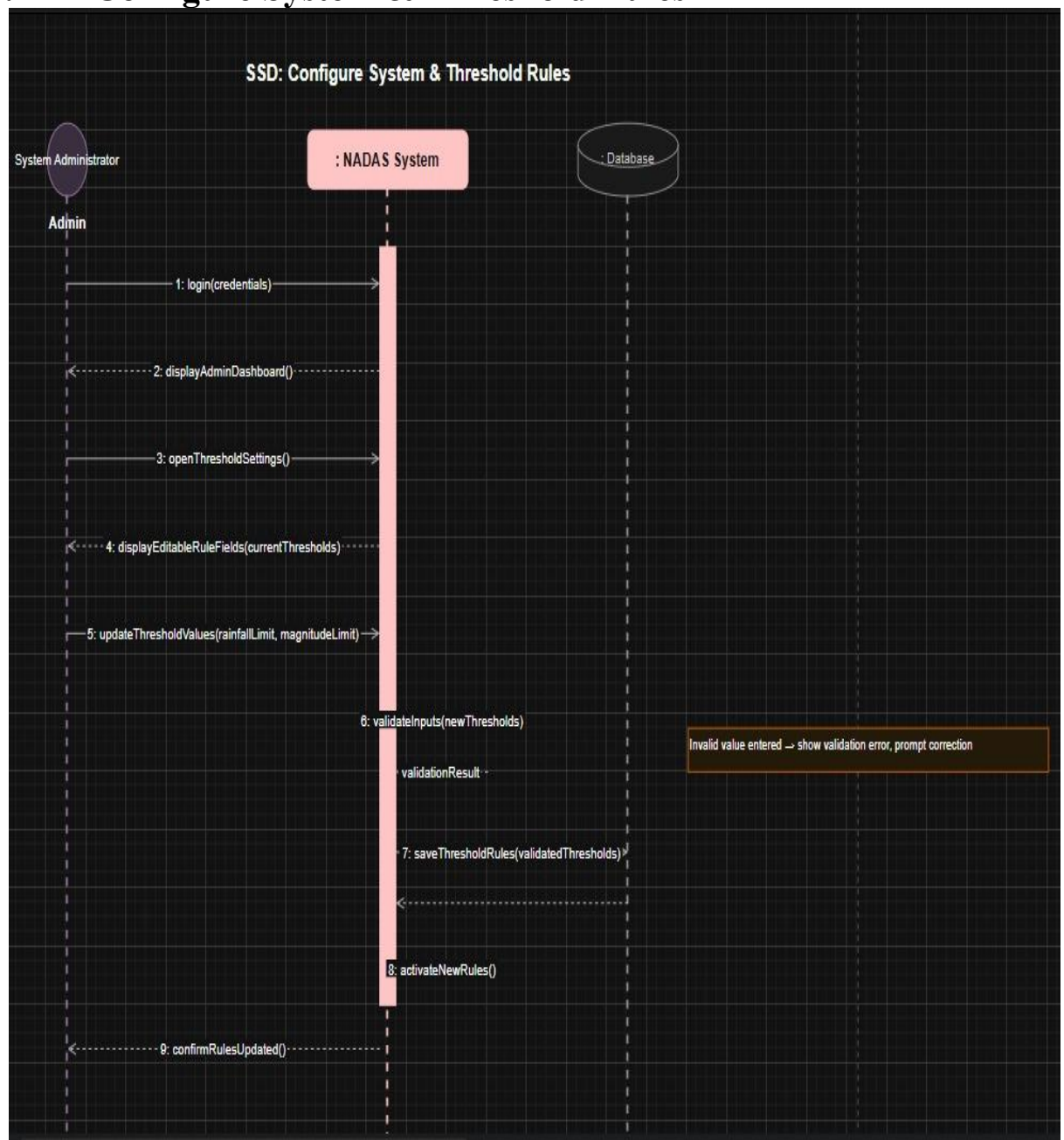
10. Generate Impact Map



11. Issue Emergency Alert

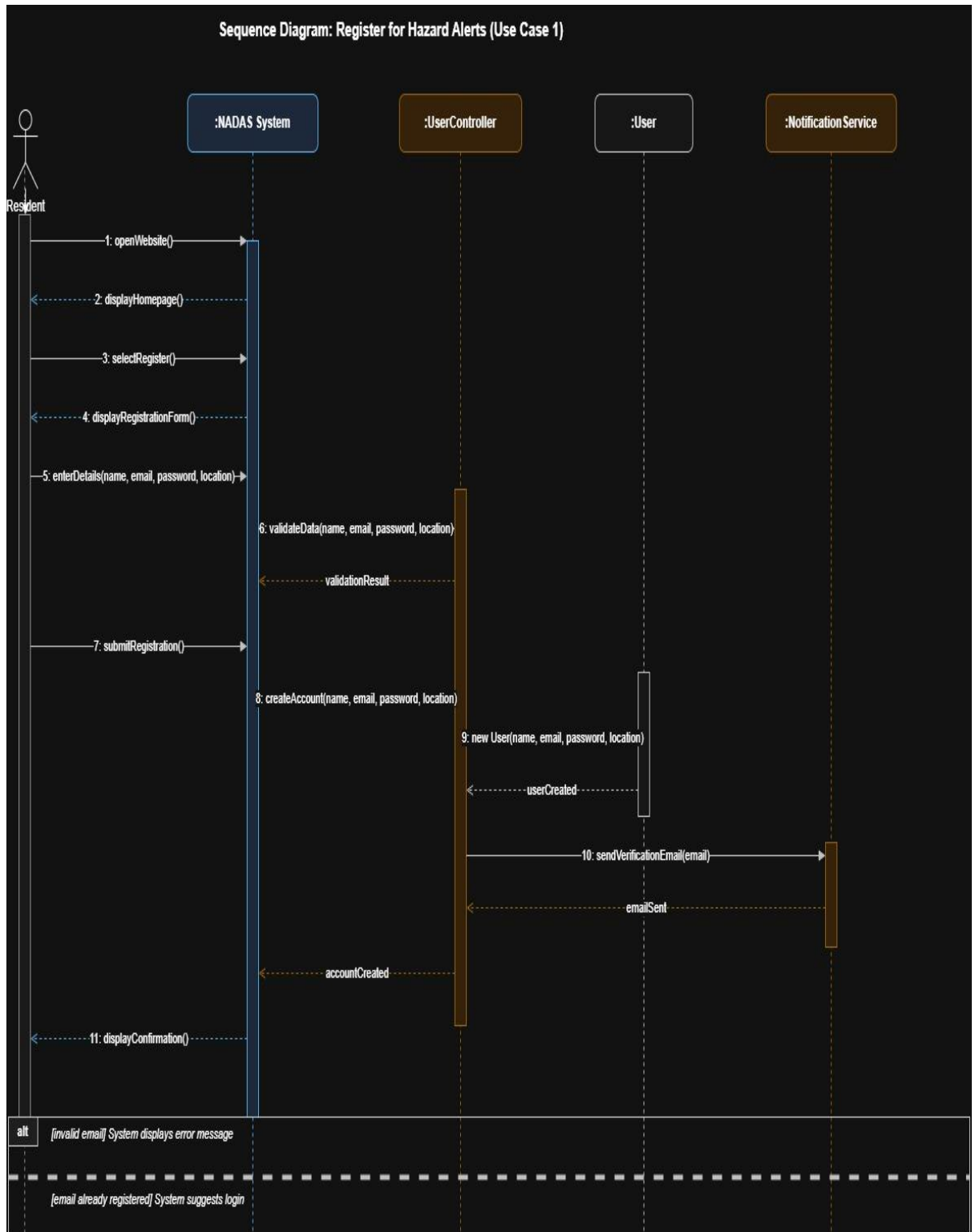


12. Configure System & Threshold Rules

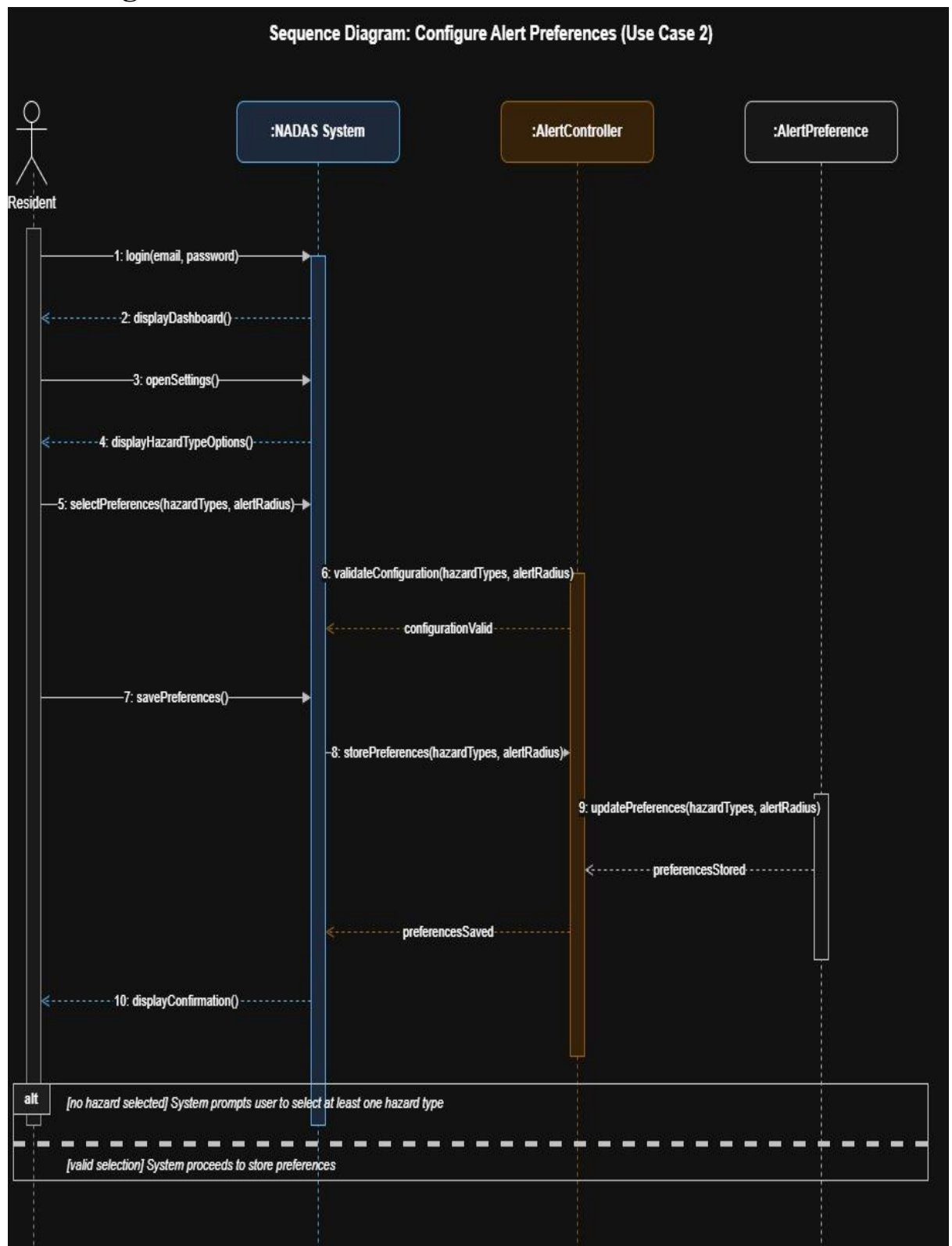


6. Sequence Diagram

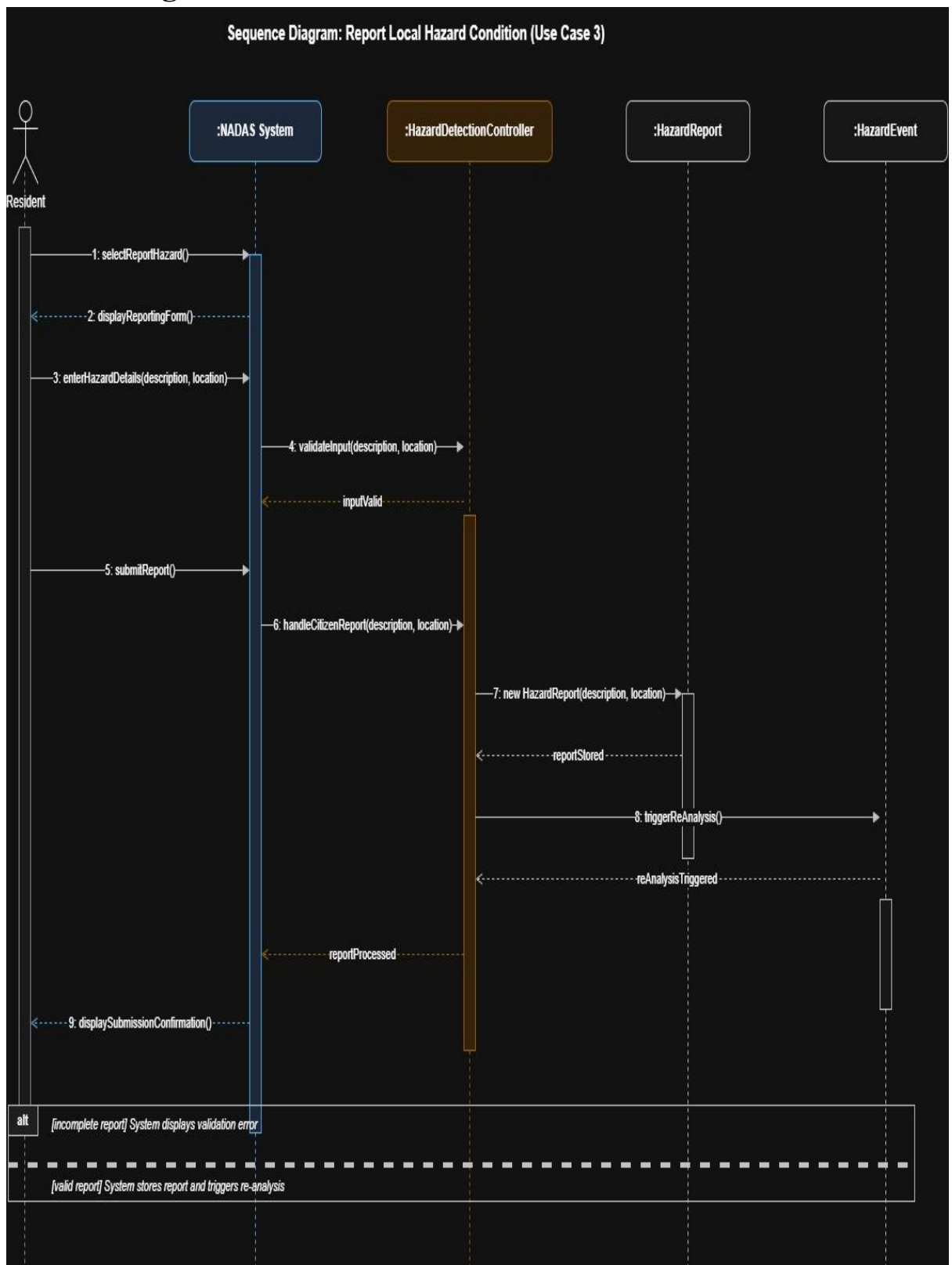
13. Monitor Weather Data



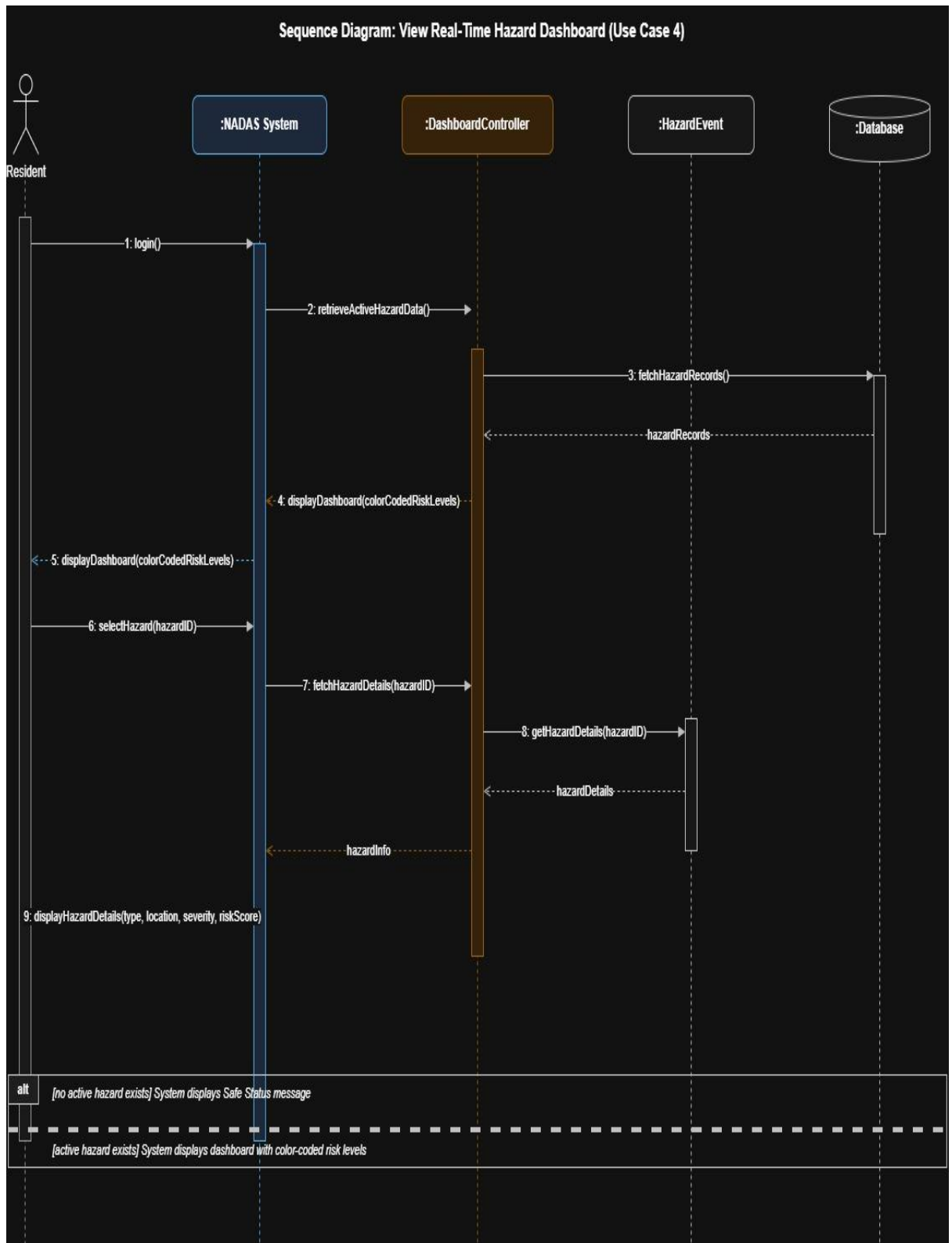
14. Register for Hazard Alerts



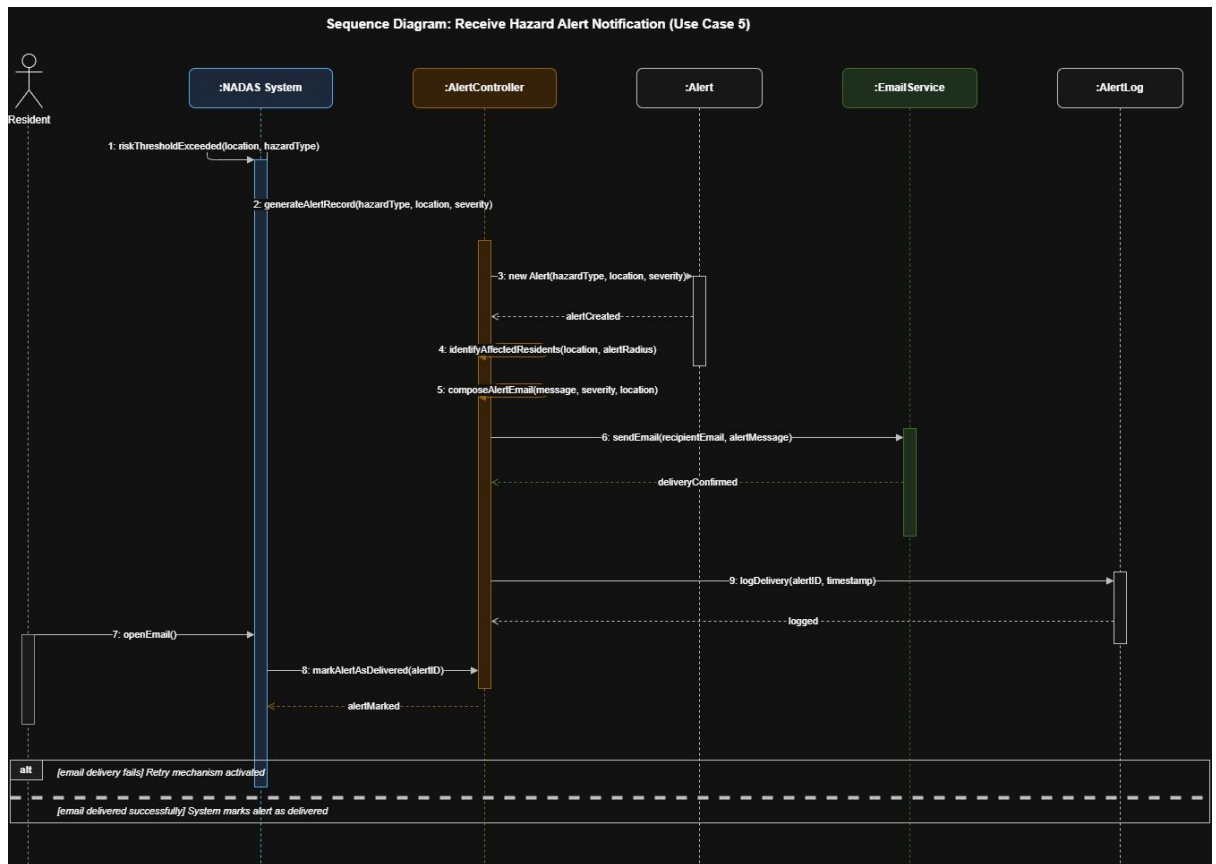
15. Configure Alert Preferences



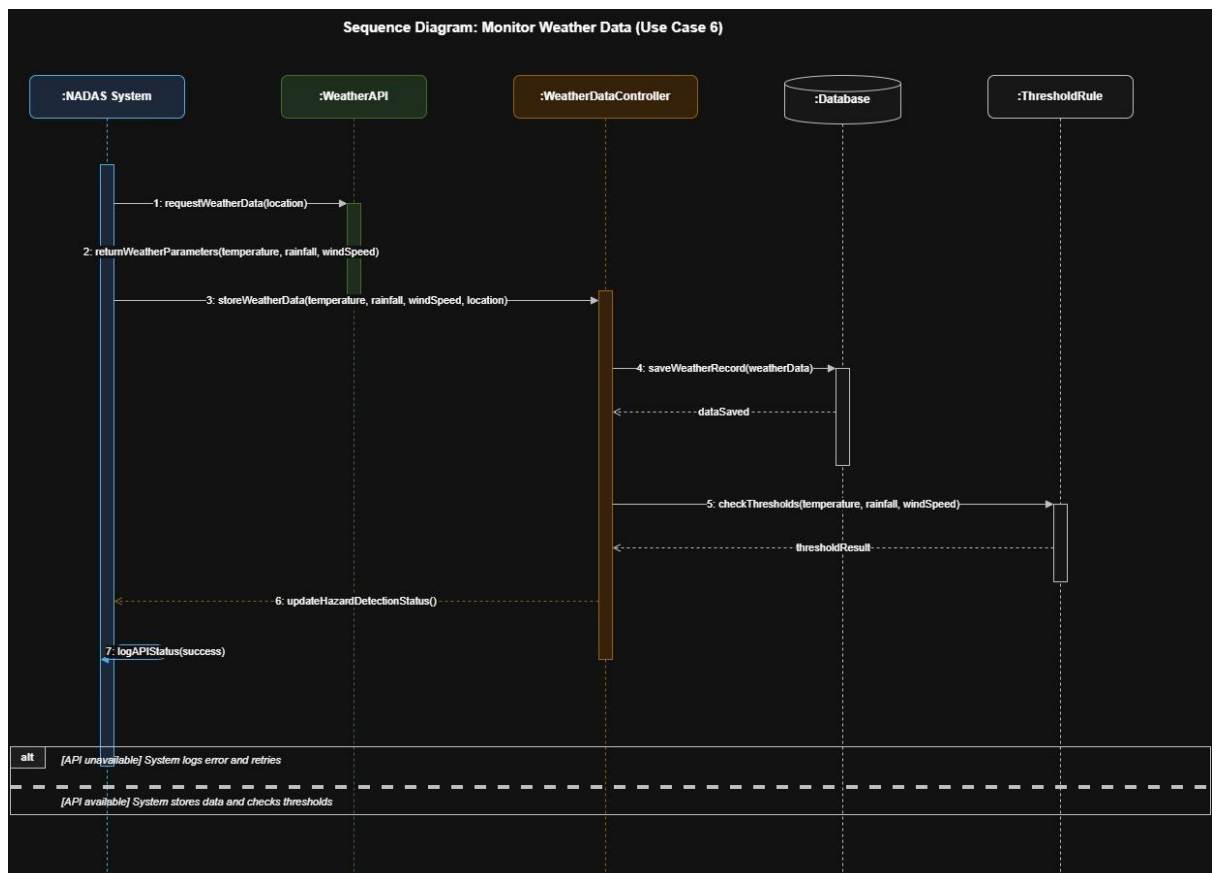
16. Report Local Hazard Condition



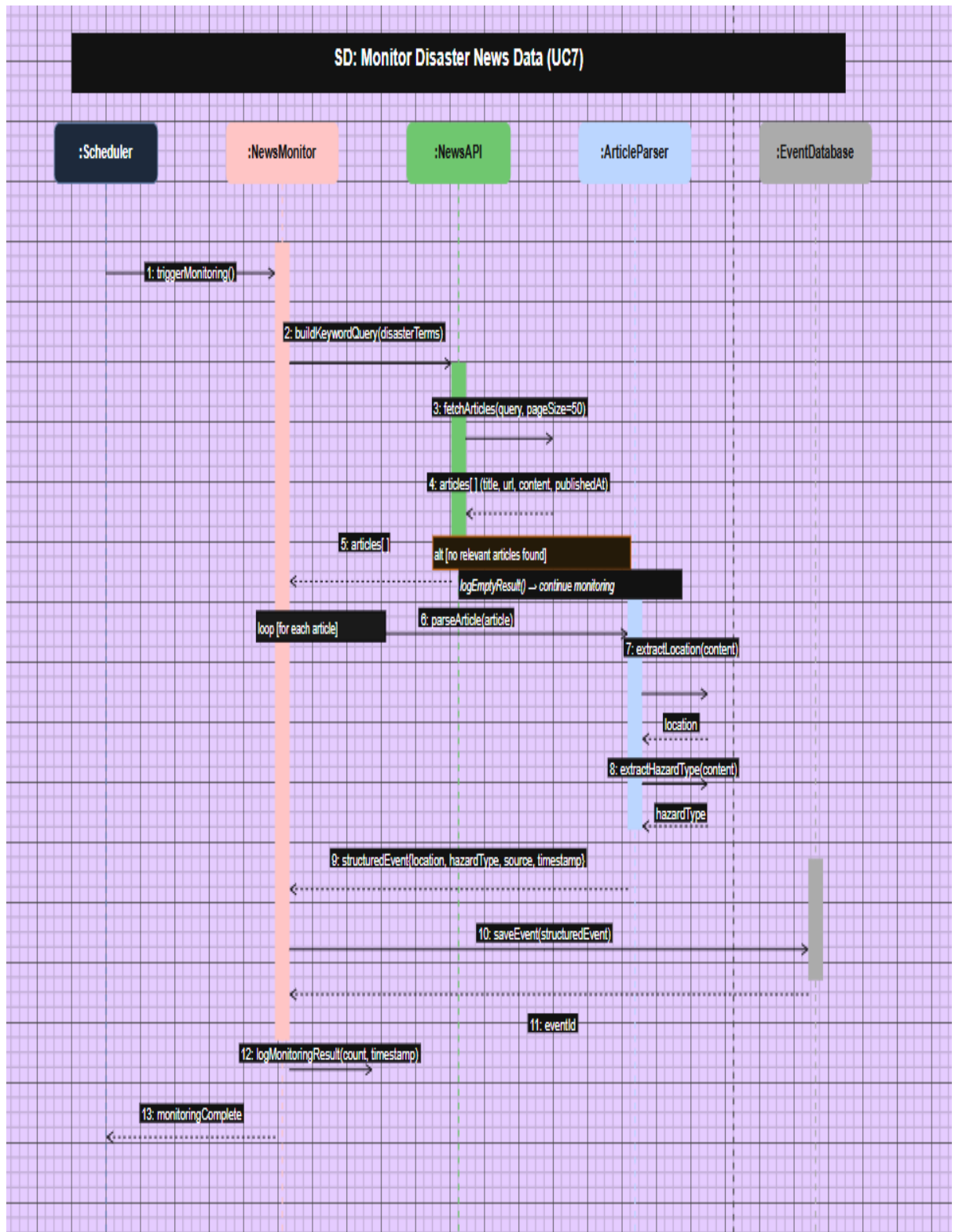
17. View Real-Time Hazard Dashboard



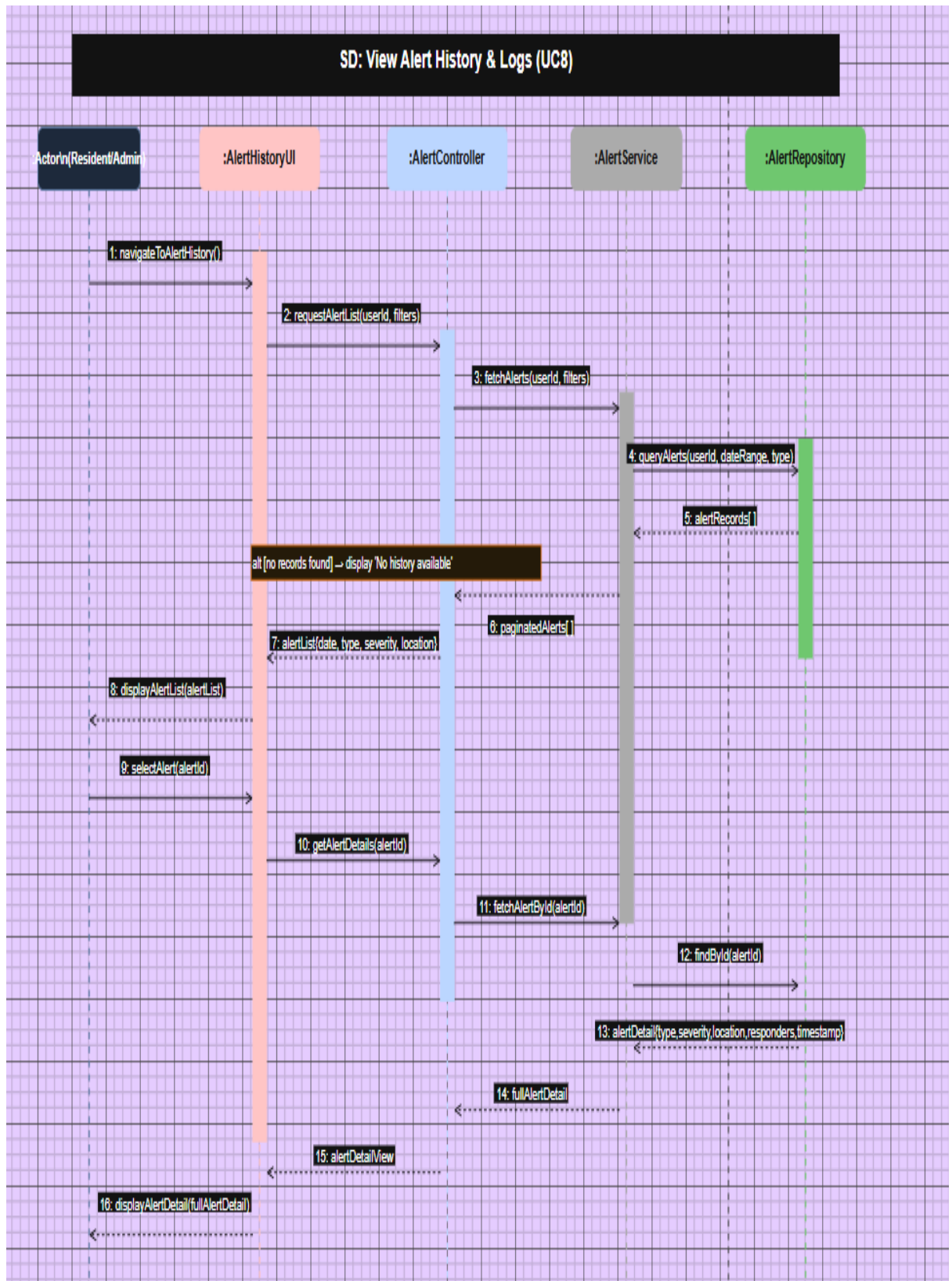
18. Receive Hazard Alert Notification



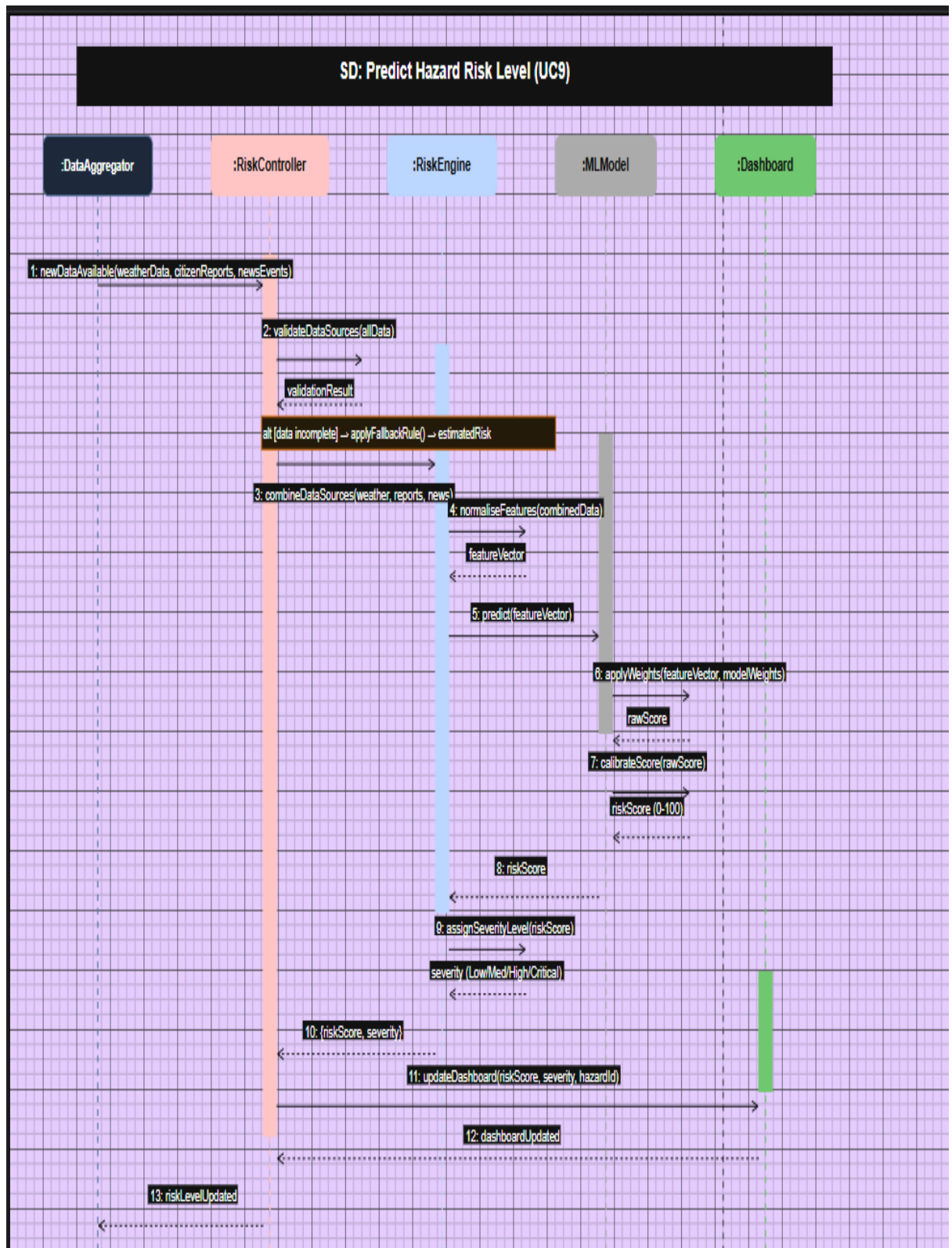
19. Monitor Disaster News Data



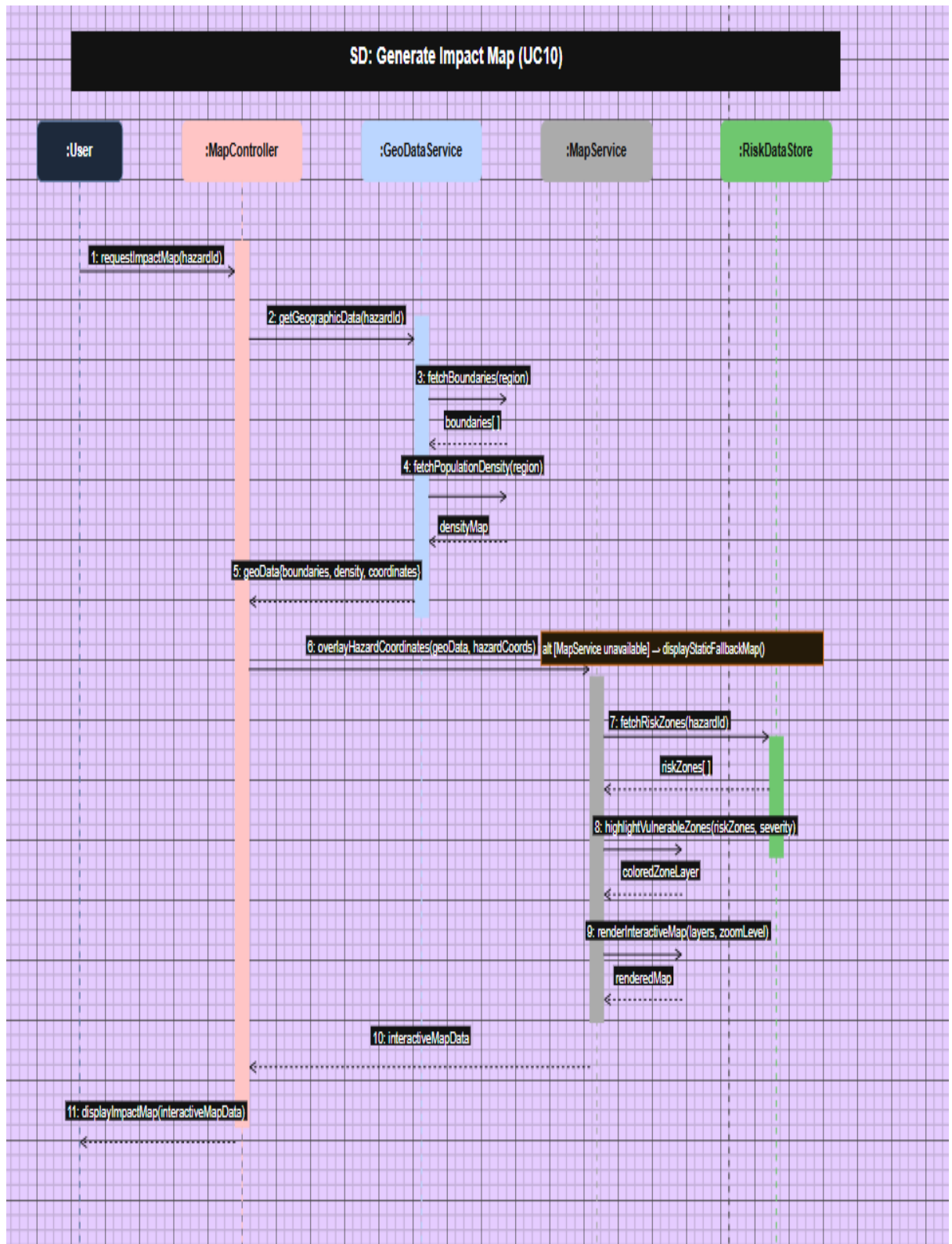
20. View Alert History & Logs



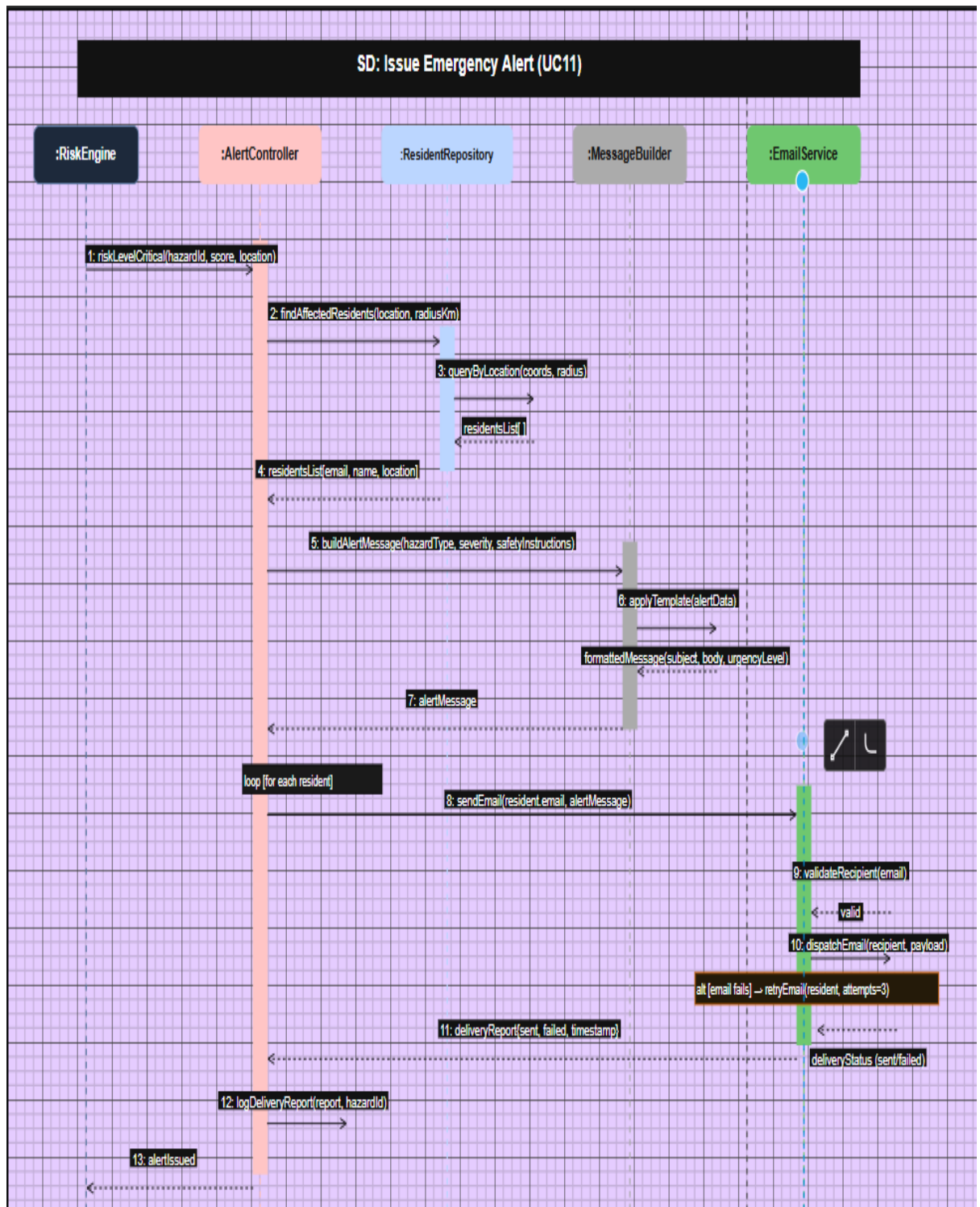
21. Predict Hazard Risk Level



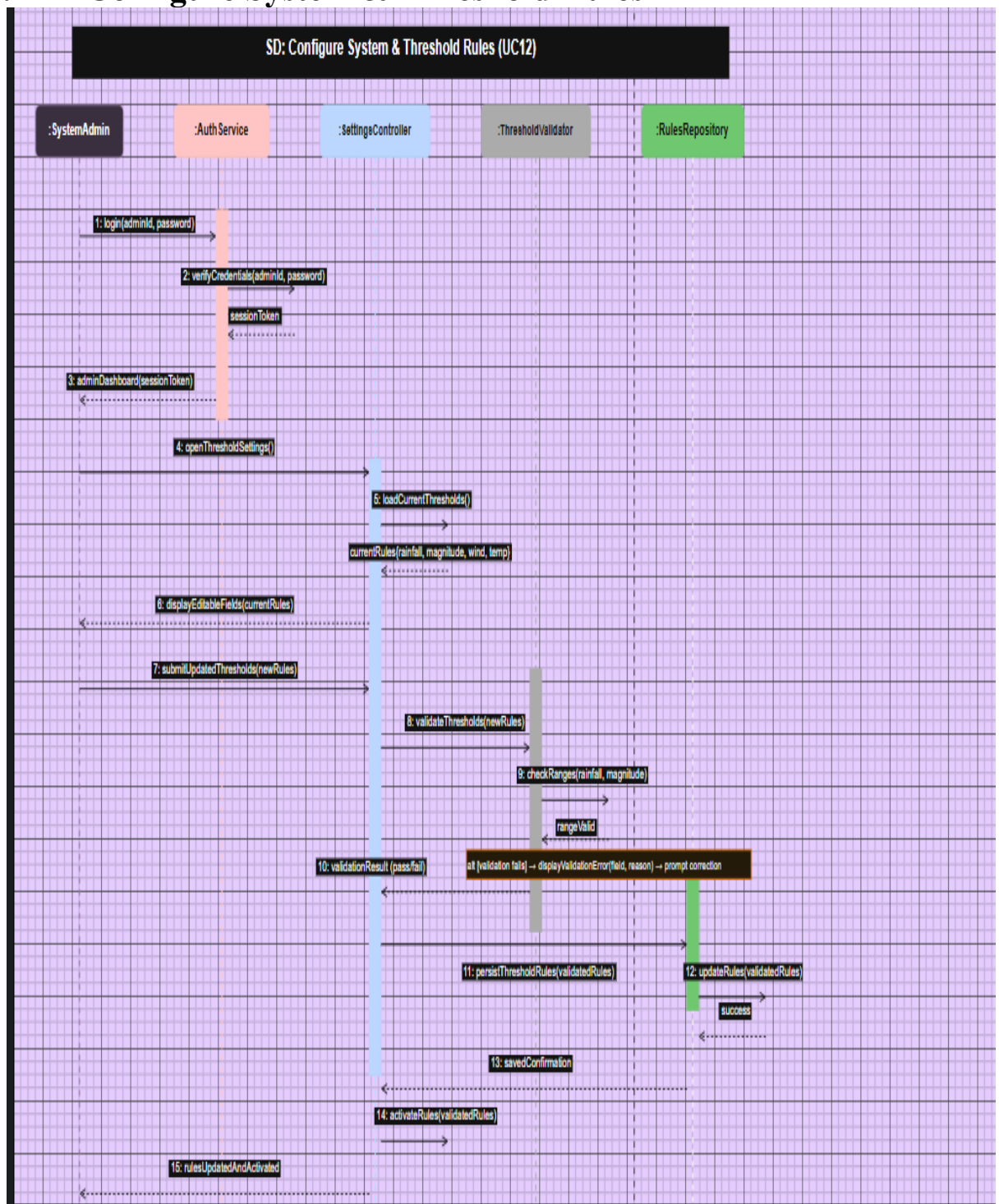
22. Generate Impact Map



23. Issue Emergency Alert



24. Configure System & Threshold Rules



7. Class Diagram

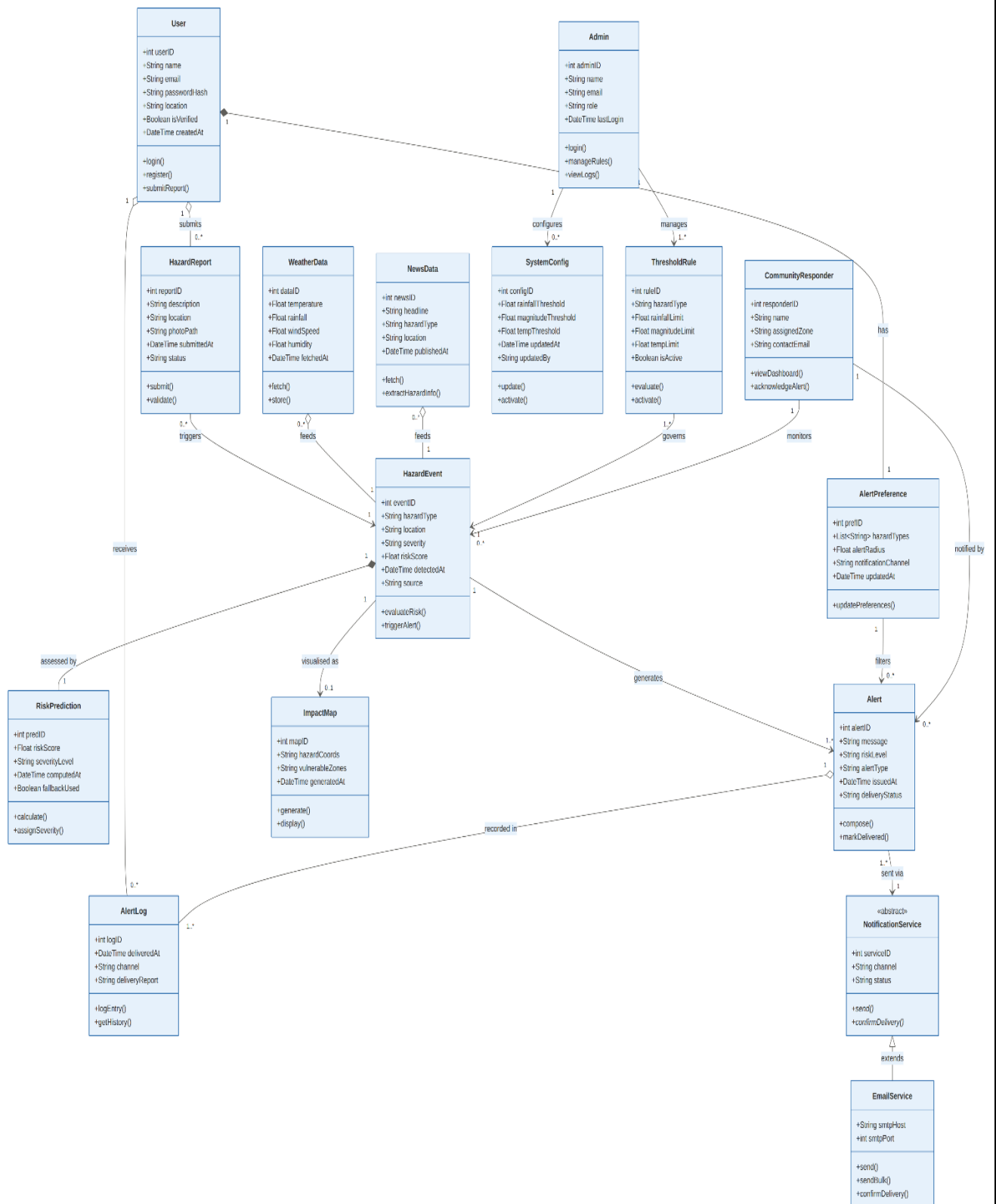


Figure 7.1: NADAS Class Diagram

7.1 Class Descriptions

3.7.1 Model Package (com.disaster.alert.model)

- User – id, name, email, password, city, alertTypes, severityLevel, emailAlerts, emergencyOnly with full getters/setters. Used throughout via SessionManager.getCurrentUser
- Admin – adminId, name, email, role with getters/setters. Authenticated via DatabaseManager.verifyAdmin().
- HazardReport – id, userId, hazardType, location, description, photoPath, status, submittedAt; getFormattedTime() formats LocalDateTime for table display.
- Alert – id, type, location, severity, message, source, timestamp; getSeverityEmoji() returns coloured emoji; getFormattedTime() for display.
- ThresholdRule – id, hazardType, magnitudeLimit, rainfallLimit, tempLimit, isActive, updatedAt.

3.7.2 Service Package (com.disaster.alert.service)

- AuthService – login(), register(), updateProfile(), changePassword() – all data access through DatabaseManager.
- AlertService – saveAlert(), getAllAlerts(), getAlertsByType(), getAlertsBySeverity(), searchAlerts().
- HazardReportService – saveReport(), updateStatus(), getReportsByUser(), getAllReports().
- EmailService – sendAlertToUsers(emails, hazardType, location, description, photoPath) using Jakarta Mail via Gmail SMTP.
- AlertFactory – create(type, location, magnitude, message) : Alert; Factory pattern with AlertType enum (EARTHQUAKE, NEWS, WEATHER, FLOOD, FIRE, USER_REPORT).

3.7.3 Database Package (com.disaster.alert.db)

- DatabaseManager – Singleton; getInstance(), connect(), createTables(), seedDefaultData(verifyAdmin(), getUserEmailsByCity(), getAllUserEmails(), getConnection(),

close().

3.7.4 Utility Package (com.disaster.alert.util)

- SceneManager – switchTo(Stage, path), switchTo(Node, path); centralises all screen navigation across the application.
- SessionManager – Singleton; login(user), logout(), isLoggedIn(), getCurrentUser().
- InputValidator – isValidEmail(), isValidPassword(), isEmpty(String...).
- ParticleNetworkCanvas – Extends Canvas; AnimationTimer renders 80 moving particles with constellation connections and mouse attraction for the Login screen background.

CursorEffectCanvas – Extends Canvas; hover-sensitive grid cells with blue glow fade effect.

8. Package Diagram

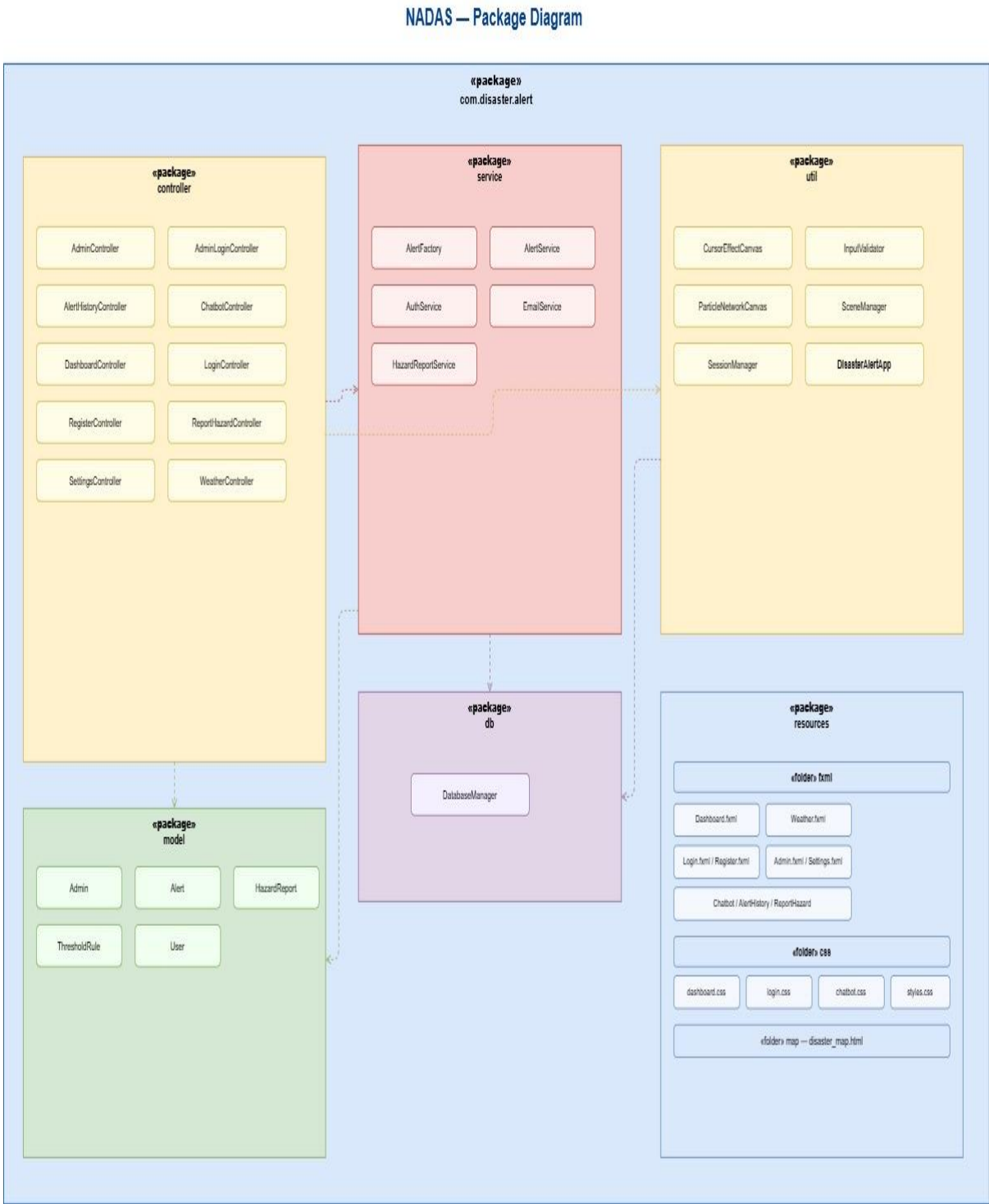


Figure 8.1: NADAS Package Diagram

8.1 Package Structure

Package	Classes
controller	AdminController, DashboardController, LoginController, RegisterController, Rep
model	Admin, Alert, HazardReport, ThresholdRule, User
service	AlertFactory, AlertService, AuthService, EmailService, HazardReportService
db	DatabaseManager
util	CursorEffectCanvas, InputValidator, ParticleNetworkCanvas, SceneManager, Sess
resources/fxml	9 FXML screen files
resources/css	dashboard.css, login.css, styles.css

9. Deployment Diagram

NADAS — Deployment Diagram

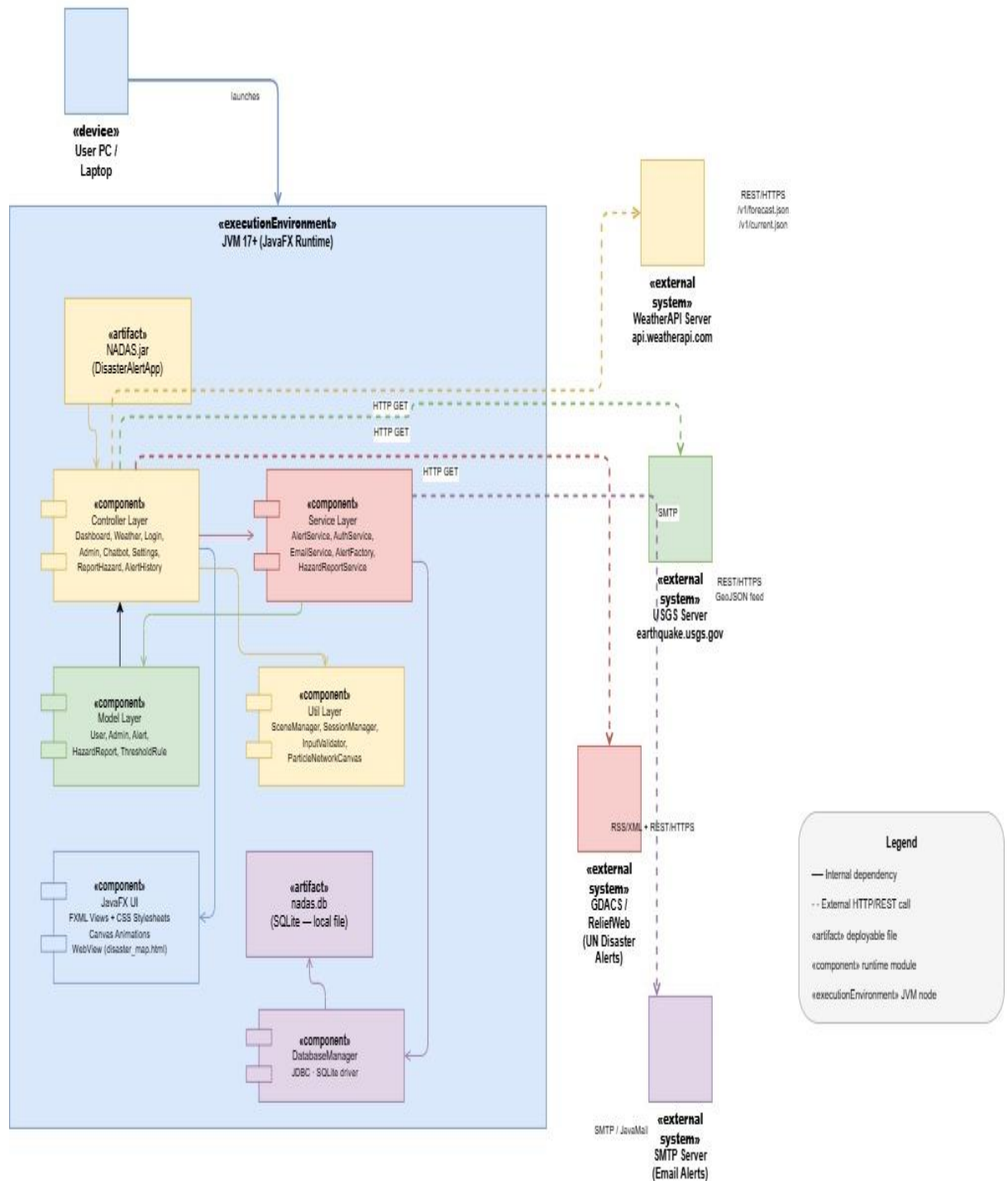


Figure 9.1: NADAS Deployment Diagram

9.1 Deployment Description

NADAS follows a single-tier client deployment with external API dependencies over HTTPS and SMTP.

Node / Component	Details
Client Machine	Windows/macOS/Linux running JDK 21 +
JavaFX 21.0.1 Application	Maven <code>javafx:run:</code>
<code>com.disaster.alert.DisasterAlertApp</code>	Local Database SQLite <code>nadas.db</code> in project root directory; auto-created on first launch
USGS API	<code>https://earthquake.usgs.gov</code> – GeoJSON over HTTPS
WeatherAPI	<code>https://api.weatherapi.com</code> – JSON over HTTPS, API key auth
GDACS API	<code>https://www.gdacs.org/xml/rss.xml</code> – XML over HTTPS
ReliefWeb	<code>https://api.reliefweb.int/v1/reports</code> – JSON
fallback Gmail SMTP	<code>smtp.gmail.com:587</code> – TLS, App Password authentication
Map Tiles	Leaflet.js 1.9.4 (CDN) rendered inside JavaFX

Table 9.1: Deployment Nodes and Communication

File Structure

Listing A.1: Maven Project Directory Structure

```
SDA_Project
|-- pom.xml
|-- nadas.
|-- db src/
    |-- main /
        |-- Dis sterAlertApp .java
        |-- a
        |-- cont roller/
            |-- Admin Controller. java
            |-- Admin Login Controller. java
            |-- AlertHistory Controller. java
            |-- ChatbotController. java
            |-- Dashboard Controller. java
            |-- Login Controller. java
            |-- RegisterController. java
            |-- ReportHazard Controller. java
            |-- SettingsController. java
            |-- WeatherController. java
        |-- mode I/
            |-- Admin .java
            |-- Alert. java
            |-- Hazard Report. java
            |-- Threshold Rule .java
            |-- User. java
        |-- serv ice /
            |-- AlertFactory .java
            |-- AlertService .java
            |-- Auth Service .java
            |-- EmailService .java
            |-- Hazard ReportService .java

        |-- Database Manager. java
        |-- util/
            |-- CursorEffectCanvas. java
            |-- InputValidator. java
            |-- Particle Network Canvas. java
```

```

|      |      |-- Scene Manager . java
|      |      '-- Session Manager . java
|      '-- resources/
|          |-- fxm /
|              1
|              |-- Admin . fxml
|              |-- AdminLogin . fxml
|              |-- AlertHistory . fxml
|              |-- Chatbot . fxml
|              |-- Dashboard . fxml
|              |-- Login . fxml
|              |-- Register . fxml
|              |-- ReportHazard . fxml
|              |-- Settings . fxml
|              '-- Weather . fxml
|          |-- css/
|              |-- dashboard . css
|              |-- login . css
|              '-- styles . css
|      '-- map/
|          '-- disaster_map . html

```